

Experiment-Overview 2026-07-26

Experiment-Overview: TDD-Workflows × Modelle × Prompt-Stile

Stand: 2026-07-26. Datenbasis: `experiments/runs/` (1127 Runs gesamt). Dieser Snapshot berichtet die **19 generischen Forschungsfragen mit 855 Runs**; die interne Workflow-Entwicklungs-Linie (13 RQs, 272 Runs) ist bewusst ausgespart.

Autor: Marco Emrich (codecentric AG) — Mit-Initiator von [EXACT Coding](#) gemeinsam mit Ferdinand Ade.

Repository: github.com/marcoemrich/agentic_coding_lab — alle Skripte, Workflow-Definitionen, Run-Artefakte und das Stylesheet sind dort öffentlich versioniert.

Über die Studie

Dieses Lab ist die empirische Validierungs-Plattform für **EXACT Coding** (EXample-guided AI-Collaborative Test-driven Coding), dessen Manuskript unter `../../../../exact-coding-book/manuscript/exact-coding.md` liegt. Die untersuchten Workflow-Varianten spannen bewusst ein Spektrum auf: von Vibe-Coding-Baselines ohne jede Test-Struktur über EXACT-konforme Setups mit expliziter Test-Liste und erzwungenem Red-Green-Refactor-Takt bis zu einer Delayed-Refactor-Kontrolle, die erst implementiert, dann nachträglich testet und genau einmal am Ende aufräumt. Dieses Spektrum ist der eigentliche Messapparat: Es erlaubt, die einzelnen EXACT-Bausteine — beispiel-getriebene Spec, vollständige Test-Liste, Test-First-Disziplin, periodisches Refactoring — gegeneinander zu isolieren, statt „mit TDD“ als undifferenziertes Paket gegen „ohne TDD“ zu stellen. Genau diese Isolation trägt den zentralen Befund dieses Snapshots, dass Korrektheit und Code-Qualität von *verschiedenen* Bausteinen getragen werden.

Stand dieses Snapshots ist der 2026-07-26. Das Lab umfasst insgesamt 1127 Runs über 32 Forschungsfragen; berichtet werden hier die **19 generischen Fragen mit 855 Runs** — jene, die Prompt-Stil, Modell, Workflow-Struktur, Kontext-Architektur und Harness gegeneinander stellen. Die 13 Fragen der internen Workflow-Entwicklungs-Linie (272 Runs), die einzelne Prompt-Bausteine gegeneinander testen, sind bewusst ausgespart: Sie sind Werkzeug-Entwicklung am Messapparat und für ein externes Publikum ohne Kenntnis der Workflow-Versionshistorie kaum lesbar. Sie liegen vollständig im Repository unter `research/workflow-dev/`.

Die Forschungs-Front hat sich gegenüber früheren Snapshots verschoben. Ob strukturiertes Vorgehen gegenüber unstrukturiertem messbar trägt, ist beantwortet und mehrfach repliziert. Offen sind heute andere Fragen: der wirksame Zeitpunkt des Refactor-Hebels (laufend in jedem Zyklus gegenüber einem einmaligen Durchgang über den gesamten Quelltext am Ende) und vor allem die Übertragbarkeit über Agent-Harnesse und Modell-Anbieter hinweg — der jüngste Zuwachs im Lab sind drei weitere Harnesse neben Claude Code und ein breites Feld an Nicht-Anthropic-Modellen.

Scope

Der Scope spannt sich über drei Achsen. (1) **Harness** — vier headless betriebene Agent-CLIs: Claude Code (2.1.170), OpenCode (1.15.10), pi (0.81.1) und cursor-cli (cursor-agent). Alle laufen **ohne Human-in-the-Loop**: Der Agent bekommt einen Prompt und arbeitet bis zum Ende oder bis zum Budget-Timeout durch, ohne dass ein Mensch zwischendurch korrigiert, nachfragt oder abbricht. (2) **Modelle** — Anthropic (Opus 4.6/4.7/4.8/5, Sonnet 4.6/5, Haiku 4.5, Fable 5, je mit und ohne Thinking) sowie über Requesty erreichbare Fremdmodelle (GLM 5.1/5.2, DeepSeek V4 Flash/Pro, Kimi K2.6/K2.7, GPT-5.6 Sol/Terra, Gemini 3.5 Flash, Mistral Medium 3.5, MiniMax M2.7/M3, Qwen3-235B, Grok 4.5, Composer 2.5). (3) **Zielsprache** — ausschließlich **TypeScript** mit pro Run identischem `pnpm/tsx/Vitest/ESLint+SonarJS`-Stack.

Die Befunde gelten für diesen Stack. Der Transfer auf andere Zielsprachen (Python, Go, Java) ist offen und wird hier nicht geprüft — insbesondere sind die Komplexitäts-Metriken über SonarJS/ESLint an das TypeScript-Ökosystem gebunden. Ebenso offen ist der Transfer auf **interaktive HITL-Setups**: Sämtliche Befunde beschreiben, was ein Agent unbeaufsichtigt produziert. Genau die Fehlermodi, die hier Korrektheit kosten (vorzeitiger Selbst-Abbruch, unvollständige Test-Listen, falsch geratene Schnittstellen-Verträge), wären in einem interaktiven Setup durch eine einzige menschliche Rückfrage abfangbar. Die Zahlen sind damit als Aussage über Autonomie-Robustheit zu lesen, nicht als Obergrenze dessen, was mit denselben Werkzeugen unter menschlicher Begleitung erreichbar ist. Die Katas sind synthetisch und klein (~30–320 Produktiv-LoC); Web-Apps, Datenbank-Code und asynchrone Systeme sind nicht abgedeckt.

AI-Hinweis

Dieser Snapshot wurde mit der `/build-overview`-Skill in **Claude Code** (Anthropic Opus 5) erstellt. Datengetriebene Sektionen — RQ-Übersichts-Tabelle, Coverage-Werte, Finding-Listen pro RQ, Reproduzierbarkeits- und Files-Tabelle — werden deterministisch aus `research/{questions,workflow-dev}/*/({README,findings}).md` via `experiments/generate-snapshot-skeleton.py` generiert. Synthese-Sektionen (Intro, Per-RQ-Paragrafen, Cross-RQ-Synthese, Limitierungen) sind vom LLM grafted und human-curated. Die Generierung ist damit vollständig nachvollziehbar.

Hauptbefunde

Vier zentrale Befunde aus den 19 Forschungsfragen — Details und Belege in §4, Cross-RQ-Synthese in §5. Kurzfassung: Die beiden EXACT-Coding-Kernbausteine wirken, aber auf **getrennte Ziele** — Example-Mapping und eine Test-Schreib-Phase tragen die Korrektheit, der erzwungene Refactor-Takt trägt die Code-Qualität. Beide zusammen ergeben den Workflow, der über beide Aufgabentypen robust bleibt:

- EXACT-Coding wirkt — die Kombo aus Example-Mapping und Tests-gegen-Spec schlägt Vibe-Coding messbar.** Auf der novel Kata (claim-office) fällt die Korrektheit (außen, `verification_pct`) von ≥ 0.96 auf 0.28, sobald ohne Test-Schreib-Phase geübt wird; Example-Mapping als Spec-Stil hebt sie zusätzlich um +48–76 Prozentpunkte gegenüber Prose. Beide Korrektheits-Hebel sind die *Spezifikation* (konkrete I/O-Beispiele) und das *Formulieren als Tests gegen die Spec* — nicht der Red-Green-Refactor-Zyklus selbst (der naive „use TDD“-Lauf erreicht ebenfalls 1.00 Korrektheit). Praktische Konsequenz: Auf novel Domänen sind konkrete I/O-Beispiele plus eine Test-Schreib-Phase die wertvollste Korrektheits-Investition.

2. **Striktes TDD verbessert die Code-Qualität messbar.** Ein Workflow mit periodischem, isoliertem Refactor-Schritt senkt auf claim-office die Komplexitäts-Spitze auf $\sim\frac{1}{2}$ und die Smell-Summe auf $\sim\frac{1}{10}$ von Vibe-Coding (`cognitive_max` 5.7 vs. 11–12, `smell_total` 1.3 vs. 12–16). Den Hebel liefert die strukturierte Refactor-Disziplin im Takt, nicht das Etikett „TDD“: Der naive Ansatz — ein Agent, der nur „use TDD“ hört und ohne erzwungenen Red-Green-Refactor-Takt sich selbst überlassen bleibt — produziert den schwersten Code der ganzen Matrix (`cognitive_max` 19.8), schlechter als gar kein TDD. Praktische Konsequenz: Für langlebigen Code zahlt sich ein Workflow mit erzwungenem Aufräum-Schritt pro Zyklus aus; eine bloße „mach es in TDD“-Aufforderung tut es nicht.
3. **Example-Mapping ist auf novel Aufgaben der dominante Korrektheits-Hebel — User-Story $\approx$ Prose.** Auf claim-office hebt Example-Mapping `verification_pct` um +48–76 Prozentpunkte gegenüber Prose (Opus 4.7: 0.21 $\rightarrow$ 0.97; Opus 4.6 no-thinking: 0.23 $\rightarrow$ 0.87; Sonnet 4.6 no-thinking: 0.23 $\rightarrow$ 0.71), weil konkrete Input/Output-Beispiele die Domänen-Mehrdeutigkeiten auflösen. User-Story wirkt praktisch identisch zu Prose ($\Delta \leq 8$ pp). Auf trainingsbekannten Katas ist der Effekt null, und der Hebel ist modell-gated: Haiku 4.5 bleibt in jedem Stil bei 0 %. Praktische Konsequenz: Beim Schreiben einer Spec für eine novel Domäne sind konkrete I/O-Paare die wertvollste Investition.
4. **Ein Hybrid-Workflow mit Skill-basiertem Red/Green im geteilten Kontext und isoliertem Refactor-Subagent ist der robuste Code-Qualitäts-Default — aber kein Workflow gewinnt universell.** Diese Architektur ist die einzige, die über beide Code-Qualitäts-Katas in den Top-2 landet (`cognitive_max` 5.7 auf claim-office gegen 19.8 bei Minimal-TDD, 6.5 auf game-of-life gegen 21.8) und zugleich die einzige mit 0 % Outlier-Rate über zehn Replikate. Zugleich tauschen phasen-isolierte und hybride Workflows je nach Modell die Plätze — derselbe Workflow erreicht auf einem Modell 0.93 und auf dem nächsten 0.67 —, und die vollständig phasen-isolierte Variante stürzt von Platz 1 auf der bekannten Kata auf Platz 8 auf der neuartigen. Praktische Konsequenz: Refactor-Phase plus Refactor-Isolation sind die Hebel — aber der beste Workflow ist auf der eigenen Modell-/Aufgaben-Kombination zu validieren.

1. Forschungsfragen-Übersicht

Forschungsfragen (Claude Code)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-prompt-correctness	Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?	aktiv	24	24/24 (100 %)	129
1.2	RQ-prompt-known-kata	Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?	aktiv	9	9/9 (100 %)	45
2.1	RQ-model-quality	Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?	aktiv	12	12/12 (100 %)	44
2.2	RQ-model-novel	Wie unterscheiden sich Fable 5, Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?	aktiv	5	5/5 (100 %)	30

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
3.1	RQ-workflow-model	Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?	aktiv	6	6/6 (100 %)	49
4.1	RQ-tdd-quality	Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualitaet aus, und macht die TDD-Striktheit einen Unterschied?	aktiv	16	16/16 (100 %)	103
4.2	RQ-tdd-correctness	Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?	aktiv	7	7/7 (100 %)	36
4.3	RQ-context	Welche Form der Kontext-Strukturierung — isolierte Subagent-Kontexte pro TDD-Phase (v4.1), ein geteilter, akkumulierter Single-Context (v5.1), ein Hybrid mit Skill-basiertem Red/Green im Shared-Context und isoliertem Refactor-Subagent (v6.1) oder ein Hybrid mit isolierten Green- und Refactor-Subagents bei Shared-Context-Test-Liste/Red (v7.1) — fuehrt zu besserer Code-Qualitaet?	aktiv	4	4/4 (100 %)	21
4.4	RQ-pocock-vs-v62	Wie schneidet der externe Matt-Pocock-TDD-Skill (v9-pocock-tdd: Single-Skill, Inline-Phasen, Tail-Refactor) auf claim-office-example-mapping gegen die interne Default-Baseline v6.2-with-why-cleaned (Multi-Command + Refactor-Subagent, Per-Cycle-Refactor) ab — auf Korrektheit, Code-Qualitaet, TDD-Disziplin und Kosten?	aktiv	2	2/2 (100 %)	11
5.1	RQ-stability	Wie stabil sind Code-Qualitaet und TDD-Disziplin pro Workflow ueber Replikate, und unter welchen Bedingungen ist n=3 als Replikat-Anzahl ausreichend?	aktiv	6	5/6 (83 %)	59

Forschungsfragen (OpenCode)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-model-quality-oc	Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle (Opus 4.7 via Portkey + vier Nicht-Anthropic-Modelle aus dem Portkey-Catalog) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?	aktiv	6	6/6 (100 %)	30
1.2	RQ-model-novel-oc	Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?	aktiv	8	8/8 (100 %)	40

Forschungsfragen (pi)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-model-quality-pi	Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v6.2.1-phase-continuation-pi-Workflow?	aktiv	10	10/10 (100 %)	50
1.2	RQ-model-novel-pi	Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v6.2-with-why-cleaned-pi-Workflow?	aktiv	15	15/15 (100 %)	75

Forschungsfragen (Cursor CLI)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-model-quality-cursor	Wie unterscheiden sich die via cursor-cli-Harness erreichbaren Modelle (Opus, Composer, Grok) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping?	aktiv	3	3/3 (100 %)	15

Forschungsfragen (Harness-übergreifend)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-harness	Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität und TDD-Disziplin aus, wenn Modell, Workflow-Intention und Prompt-Stil konstant gehalten werden?	aktiv	6	6/6 (100 %)	38
1.2	RQ-harness-requesty	Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität, TDD-Disziplin und Kosten aus, wenn Modell (opus-4-8 über Requesty), Workflow-Intention und Prompt-Stil konstant gehalten werden?	aktiv	8	8/8 (100 %)	40
1.3	RQ-cost-sol-pi-vs-opus-cc	Wie viel günstiger ist das GPT-Modell gpt-5-6-sol auf dem pi-Harness gegenüber opus-4-8 auf Claude Code — bei gleichem Prompt-Stil und outcome-äquivalentem TDD-Workflow, über beide Katas?	aktiv	4	4/4 (100 %)	20
1.4	RQ-model-quality-cc-vs-pi	Unterscheidet sich das Code-Qualitäts-Profil von Opus (opus-4-8) zwischen dem Claude-Code- und dem pi-Harness, je mit und ohne Thinking, bei konstanter Workflow-Generation (v6.2)?	aktiv	4	4/4 (100 %)	20

2. Experiment-Design

2.1 Variablen

Workflow — sechs Generationen (Details: [research/workflow-dev/workflow-construction.md](#) — Inventar):

Workflow	Aufbau	TDD-Strenge
v1-oneshot	"Implementiere X."	keine
v2-iterative	"Plane Schritt für Schritt, dann implementiere."	keine
v3-basic-tdd	Inline TDD, kein Skill/Subagent (Self-Reporting)	minimal
v4-exact-subagents	Eigener Subagent pro Phase (Predictor + Red/Green/Refactor), fresh context	strikt, multi-context
v4.1-testlist-scope-fix	v4 mit Test-List-Scope-Patch	strikt, multi-context
v5-exact-single-context	Alle Phasen in einer Konversation, gleiches Phasen-Skript	strikt, single-context
v5.1-testlist-scope-fix	v5 mit Test-List-Scope-Patch (an v4.1 angeglichen)	strikt, single-context
v6-hybrid	Hybrid: inline TDD + nur Refactor als Subagent	strikt, hybrid
v6.1-hybrid-testlist-scope-fix	v6-hybrid mit Test-List-Scope-Patch (aktuelle Default-Basis)	strikt, hybrid
v6.1-no-pep	v6.1 ohne Pep-Talks (RQ-pep-Replikation)	strikt, hybrid
v7-hybrid-green-refactor	Wie v6, aber green und refactor als Subagent	strikt, mehr Isolation

Workflow	Aufbau	TDD-Streng
v7.1-hybrid-green-refactor-testlist-scope-fix	v7 mit Test-List-Scope-Patch	strikt, mehr Isolation
v8a-delayed-refactor-agent	Oneshot → nachträgliche Tests → einmaliger End-Refactor-Agent (refactor.md aus v6.5.4)	delayed-refactor
v8b-delayed-refactor-native	Wie v8a, aber nativer Inline-Refactor im v3-Stil, kein Agent	delayed-refactor

Konfiguration: `experiments/workflows/<variant>/` und `.claude/agents/` und `.claude/rules/`. Archivierte Varianten (v5.1-minimized, v6.2–v6.6, v6.5.x-Audits) liegen unter `experiments/workflows/_archive/`.

Workflow-Mechanik im Detail. Die sechs Generationen sind nicht nur eine Skala "mehr/weniger TDD", sondern eine systematische Variation der EXACT-Coding-Bausteine (Test-Liste, Red, Green, Refactor) und ihrer Kontext-Architektur:

- **v1-oneshot / v2-iterative — Vibe-Coding-Baselines (kein TDD).** Ein einzelner Agent liest die Anforderungen und schreibt Code in einem Schritt (v1) oder mit explizitem Plan/Checkliste (v2); Tests werden erst nachträglich auf Basis des Example Mappings hinzugefügt. Dient als Messlatte für den Wert von TDD selbst (siehe `experiments/workflows/v1-oneshot/.claude/rules/experiment-mode.md`).
- **v3-basic-tdd — Minimal-TDD ohne Struktur.** Ein einziger Agent mit minimaler Anweisung "use TDD" — keine Phasen-Prompts, keine Subagents. Claude entscheidet selbst, wie es den TDD-Prozess strukturiert. Misst, wie weit eine reine Aufforderung trägt (`v3-basic-tdd/.claude/rules/experiment-mode.md`).
- **v4-exact-subagents / v4.1-testlist-scope-fix — Strict TDD, multi-context.** Jede TDD-Phase läuft als spezialisierter Subagent in **isoliertem Kontext** (`Task(subagent_type: "red")` etc.): `test-list` → `red` → `green` → `refactor`. Hypothese: isolierte Kontexte erzwingen Disziplin, können aber Zustand zwischen Phasen verlieren. v4.1 ergänzt im `test-list`-Subagent die Pflicht "Cover every spec example" — schließt den dominanten Failure-Mode auf novel Katas (unvollständige Test-Liste) auf Opus 4.7.
- **v5-exact-single-context / v5.1-testlist-scope-fix — Strict TDD, single-context.** Identisches Phasen-Skript wie v4, aber alle Phasen laufen im **gleichen Kontext** als Skill-Calls (`Skill(skill: "red")` etc.) statt als Subagents. Hypothese: shared context erhält den Zustand, kann aber zu Disziplin-Verlust führen. v5.1 spiegelt v4.1 mit dem identischen Test-List-Scope-Patch.
- **v6-hybrid / v6.1-hybrid-testlist-scope-fix — Hybrid mit isoliertem Refactor.** Red und Green laufen inline als Skills im Shared-Context (wie v5), Refactor läuft als isolierter Subagent (wie v4). Hypothese: kombiniert die Spec-Kohärenz des Single-Context mit der Disziplin-Schärfung der Subagent-Isolation am kritischsten Punkt (Refactor). v6.1 ist die aktuelle Default-Basis und Champion über mehrere RQs. `v6.1-no-pep` testet die Reduktion psychologischer Begründungen in Red/Green.
- **v7-hybrid-green-refactor / v7.1-....-testlist-scope-fix — Hybrid mit isoliertem Green + Refactor.** Zusätzlich zur Refactor-Isolation aus v6 läuft auch Green als isolierter Subagent. Test-Liste und Red bleiben im Shared-Context. Prüft, ob mehr Isolation gleich besser ist (Pareto-dominiert von v6 auf game-of-life: spart Tokens, verliert Qualität und Korrektheit).
- **v8a-delayed-refactor-agent / v8b-delayed-refactor-native — Delayed-Refactor-Kontrolle.** Drei sequentielle Phasen ohne TDD-Cycles: (1) Oneshot-Implementation, (2) nachträgliche Tests gegen `prompt.md` mit Coverage-Pflicht, (3) ein einmaliger End-Refactor. v8a nutzt den `refactor.md`-Subagent aus v6.5.4 (APP + Naming + Mandatory-Attempt), v8b einen nativen Inline-Refactor im v3-Stil ohne Agent. Dient als Kontroll-Achse für die Hypothese "periodisches TDD-Refactor schlägt End-Refactor nach Vibe-Coding".

Tiefere Mechanik-Diskussion, Inventar der aktiven v6.1-Reduktionslinie und tragende RQ-Befunde stehen in `research/workflow-dev/workflow-construction.md`. Welche Marker das Parsing der TDD-Metriken treibt, dokumentiert `experiments/workflows/MARKERS.md`. Die archivierte v6.5.x-Linie liegt in `experiments/workflows/_archive/` und `research/_archive/workflow-dev-v1/`.

Modell × Thinking (Lab-Varianten-IDs aus `MODEL_CONFIGS` in `experiments/docker/run-batch.sh`):

Lab-Varianten-ID	API-ID	Thinking	Routing
opus-4-7	claude-opus-4-7	Adaptive	Direct
opus-4-7-no-thinking	claude-opus-4-7	aus	Direct
sonnet-4-6	claude-sonnet-4-6	Extended	Direct
sonnet-4-6-no-thinking	claude-sonnet-4-6	aus	Direct
haiku-4-5	claude-haiku-4-5-20251001	Extended	Direct
haiku-4-5-no-thinking	claude-haiku-4-5-20251001	aus	Direct
opus-4-7-portkey	@vertex-eu-global/anthropic.claude-opus-4-7	Adaptive	Portkey
opus-4-7-portkey-no-thinking	@vertex-eu-global/anthropic.claude-opus-4-7	aus	Portkey
opus-4-6-portkey	@vertex-ai/anthropic.claude-opus-4-6	Adaptive	Portkey
opus-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-opus-4-6	aus	Portkey
sonnet-4-6-portkey	@vertex-ai/anthropic.claude-sonnet-4-6	Extended	Portkey
sonnet-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-sonnet-4-6	aus	Portkey
haiku-4-5-portkey	@vertex-ai/anthropic.claude-haiku-4-5@20251001	Extended	Portkey
haiku-4-5-portkey-no-thinking	@vertex-ai/anthropic.claude-haiku-4-5@20251001	aus	Portkey

Direct- und Portkey-Routings desselben Modells sind getrennte Varianten und werden nur per expliziter `controls.model: {any: [...]}`-Klausel pro RQ als gemeinsame Zelle gewertet.

Kata × Prompt-Stil (aktive Katas in `experiments/katas/`):

Kata-Basis	Prompt-Stile	Verifikations-Suite	Hinweis
game-of-life	prose, example-mapping, user-story	nein	Code-Qualität, groß (~40 LoC), vitest-basiert
game-of-life-cli	prose, example-mapping, user-story	ja	CLI-Variante mit externer Akzeptanz-Suite
mars-rover	prose, example-mapping, user-story	nein	mittel (~30 LoC), vitest-basiert
claim-office	prose, example-mapping, user-story	ja	Korrektheit, novel Versicherungs-Domäne (HPSMV/MHPCO), 15 Szenarien
claim-office-lite	prose, example-mapping, user-story	ja	Reduzierte claim-office-Variante (10 Szenarien) für Code-Qualitäts-Research

Prompt-Stile: - **prose**: Beschreibung der Regeln in Prosa, keine Test-Beispiele. - **example-mapping**: Regel + 1–3 konkrete Input/Output-Beispiele pro Regel. - **user-story**: "Als X möchte ich Y, damit Z" — Beschreibung ohne Beispiele.

2.2 Workflow → Prompt-Mapping

Aus methodischer Symmetrie (siehe Top- README.md , Abschnitt 'Methodology constraints'):

Workflow	erlaubte Prompt-Stile	Begründung
v1, v2	nur prose	Test-Beispiele in example-mapping wären für Non-TDD-Workflows ein verstecktes Test-Geschenk → unfair gegenüber den TDD-Workflows.
v3, v4(.1), v5(.1), v6(.1), v7(.1), v8a/b	alle drei	Beispiele dienen als natürliche Test-Cases — für TDD-/Refactor-Workflows ist das das Idealbild der Aufgabe.

3. Methodik

Die Pipeline-Beschreibung wurde gegen experiments/docker/Dockerfile , experiments/analyze-run.sh und experiments/aggregate-by-query.py geprüft und in zwei Punkten gegenüber älteren Snapshots korrigiert: Der Container pinnt inzwischen **claude-code 2.1.170** (nicht 2.1.107) und enthält zusätzlich die drei weiteren Harnesse (OpenCode, pi, cursor-agent). Schritt 4 der Pipeline ist entsprechend harness-abhängig — `claude --print` gilt nur für den Claude-Code-Arm; OpenCode, pi und cursor-cli werden mit ihren jeweiligen Headless-Aufrufen gestartet. Die Analyse-Schritte 5–6 sind für alle Harnesse identisch, da `analyze_transcript.py` pro Harness einen eigenen Transcript-Parser besitzt und in dieselbe `metrics.json`-Struktur schreibt.

3.1 Run-Pipeline

- Container-Image `docker-batch` (Node 22 slim, `claude-code 2.1.170` / `opencode 1.15.10` / `pi 0.81.1` / `cursor-agent` gepinnt) wird gestartet.
- Run-Dir `runs/<timestamp>_<kata>_<workflow>_<model>/` wird angelegt; Workflow-Konfig (`.claude/agents/` , `.claude/rules/`) und Kata-Prompt (`prompt.md`) hinein kopiert.
- pnpm-Workspace mit TypeScript, Vitest, ESLint+SonarJS aufgesetzt.
- Der jeweilige Harness läuft headless, ohne HITL (Claude Code via `claude --print "$(< prompt.md)";` OpenCode, pi und cursor-agent über ihre entsprechenden Headless-Aufrufe).
- `analyze-run.sh` schreibt `metrics.json` und `analysis-report.md`.
- `aggregate-by-query.py <RQ>/` baut `runs.csv` und `summary.md` pro RQ.

3.2 Erfasste Metriken

Verbindliche Termini (Spalte "Term") sind im Top- README.md definiert — alternative Synonyme sind verboten, weil sie kollidieren oder mehrdeutig sind. Volle Metrik-Tabelle inklusive externer Referenzen (Stryker, SonarJS, McCabe-Paper etc.) im README Abschnitt "Metrics".

Korrektheit

Metrik	Term	Was misst es	Richtung
tests_passing	Korrektheit (innen)	Boolean: laufen die vom Agenten geschriebenen Vitest-Tests am Ende des Runs grün?	true = besser
verification_pct	Korrektheit (außen)	Anteil bestandener Verifikations-Szenarien aus einer externen Acceptance-Suite, die der Agent nie zu sehen bekommt (0.0–1.0). Nur für CLI-Katas mit <code><basename>-verification/</code> -Verzeichnis.	höher = besser

Effizienz

Metrik	Term	Was misst es	Richtung
duration_seconds	—	Wallclock-Sekunden des <code>claude --print -Runs</code> inkl. aller Subagent-Spawns	kleiner = besser
total_tokens	—	Summe aller Tokens (Input + Output + Cache) über alle Subagent-Spawns hinweg	kleiner = besser

Metrik	Term	Was misst es	Richtung
context_utilization_pct	—	Finale Context-Window-Auslastung im Main-Context, in Prozent	informativ

Code-Mass & Umfang

Metrik	Term	Was misst es	Richtung
code_mass	Code-Mass (APP)	Gewichtete Summe der Produktiv-Code-Konstrukte (Konstanten, Invocations, Conditionals, Loops, Assignments — gestaffelte Gewichte nach Komplexität) gemäß <i>Absolute Priority Premise</i> (Micah Martin). Vergleicht Implementationen objektiver als reine LoC.	kleiner = besser
cc_loc	Produktiv-LoC	Produktiv-LoC ohne Tests, aus dem Clean-Code-Reporter	kleiner = besser (bei gleicher Korrektheit)
test_lines	Test-LoC	Anzahl Zeilen Test-Code (Vitest)	informativ
tests_total	—	Anzahl vom Agenten geschriebener Tests	informativ

Code-Qualität (ESLint + SonarJS)

Metrik	Term	Was misst es	Richtung
cc_longest_function	Spitzen-Komplexität	Längste Funktion in Zeilen — Proxy für die schlechteste Stelle im Code	kleiner = besser
cc_avg_loc_per_function	—	Mittlere Funktionsgröße in Zeilen	kleiner = besser
cc_median_loc_per_function	—	Median-Funktionsgröße (robust gegen einzelne lange Outlier)	kleiner = besser
cc_functions	—	Anzahl Funktionen	informativ
mccabe_max / mccabe_avg / mccabe_high_count	—	McCabe Cyclomatic Complexity pro Funktion: Maximum, Mittel, Anzahl über Schwellwert. Klassische Verzweigungs-Metrik.	kleiner = besser
cognitive_max / cognitive_avg / cognitive_high_count	—	SonarSource Cognitive Complexity pro Funktion: gewichtet Nesting und Control-Flow-Breaks stärker als McCabe, näher an menschlich wahrgenommener Komplexität. Diagnostisch tragende Hauptmetrik dieser Studie.	kleiner = besser
smell_total	Smell-Summe	Aggregierte Anzahl ESLint+SonarJS-Verstöße über alle Regeln	kleiner = besser
smell_complexity	—	Subset von <code>smell_total</code> : cognitive-complexity, max-depth, max-lines-per-function, max-params, no-nested-switch	kleiner = besser
smell_magic_numbers	—	Subset: ESLint <code>no-magic-numbers</code> -Verstöße	kleiner = besser
smell_duplication	—	Subset: SonarJS <code>no-duplicate-string</code> und verwandte Duplikations-Regeln	kleiner = besser
smell_code_quality	—	Subset: SonarJS <code>no-collapsible-if</code> , <code>no-redundant-jump</code> etc., plus ESLint <code>no-unreachable</code>	kleiner = besser
coverage_statements_pct	—	Statement-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser
coverage_branches_pct	—	Branch-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser

Test-Stärke

Metrik	Term	Was misst es	Richtung
mutation_score	Mutation-Score	Anteil der Stryker-Mutanten, die von der Test-Suite des Agenten gekillt werden (0.0–1.0): $(\text{Killed} + \text{Timeout}) / (\text{Killed} + \text{Survived} + \text{Timeout} + \text{NoCoverage})$. Hidden Metric — kommt in keinem Workflow-Prompt vor, daher Goodhart-resistent. Opt-in per RQ, nur für <code>tests_passing = true</code> .	höher = besser

TDD-Disziplin (aus `transcript.jsonl` + `transcript-subagents/`; getrieben von vier Markern in `experiments/workflows/MARKERS.md` — fehlt ein Marker, fällt die zugehörige Metrik still auf null)

Metrik	Term	Was misst es	Richtung
cycle_count	—	Anzahl Red-Green-Refactor-Zyklen pro Run	informativ (höher = feiner zerlegt)

Metrik	Term	Was misst es	Richtung
refactorings_applied	—	Anzahl explizit angewandter Refactoring-Schritte	höher = besser (bei TDD-Workflows)
predictions_correct / predictions_total	—	Red-Phase-Vorhersagen über Compile-/Runtime-Failure: korrekt vs. gesamt. Tiefe des Code-Verständnisses des Agenten. Pro Cycle 1–2 Predictions je nach Workflow.	Quote höher = besser
tests_passed_immediately	—	Anzahl Tests, die in der Red-Phase bereits grün waren — Indikator für Over-Implementation in vorherigen Green-Phasen	kleiner = besser
avg_red_seconds / avg_green_seconds / avg_refactor_seconds	—	Mittlere Phasendauer pro Cycle	informativ

3.3 Bewertungsgrundsätze

- **Korrektheit zuerst:** ein Run mit `tests_passing=false` zählt nicht als gleichwertige Lösung.
- **Pro Kata aggregieren:** Workflow×Modell-Tabellen werden ausschließlich pro Kata gebildet.
- **Effekt-Schwelle:** Bei n=1 pro Zelle gelten nur Differenzen mit Faktor ≥ 2 oder klar getrennten σ -Bändern als belastbar.

4. Ergebnisse

Forschungsfragen (Claude Code)

1.1 RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?

Datenbasis: 129 Runs · Coverage: 24/24 Zellen (100 %) bei min_replicates=5.

Korrektheit (außen) nach Modell × Prompt-Stil × Thinking (höher = besser; 🏆 = bester Stil pro Zeile):

Modell	Modus	prose	example-mapping	user-story
opus-4-7	+thinking	0.29	0.95 🏆	0.21
opus-4-7	-thinking	0.21	0.97 🏆	0.13
opus-4-6	+thinking	0.24	0.72 🏆	0.22
opus-4-6	-thinking	0.23	0.87 🏆	0.18
sonnet-4-6	+thinking	0.21	0.35 🏆	—
sonnet-4-6	-thinking	0.23	0.71 🏆	0.17
haiku-4-5	+thinking	0.00	0.00	0.01
haiku-4-5	-thinking	0.00	0.00	0.00

Werte: mean(verification_pct), je n=5 (opus-4-6 EM n=4; opus-4-7 -thinking EM n=9). Haiku-Zeilen ohne Sieger — alle Werte ~0.

Befunde:

- **F-prompt-correctness.1** — Schwache Modelle scheitern unabhängig vom Prompt-Stil
- **F-prompt-correctness.2** — Example-Mapping hebt Korrektheit massiv
- **F-prompt-correctness.3** — Thinking schadet bei Example-Mapping (Sonnet > Opus)
- **F-prompt-correctness.4** — User-Story ≈ Prose, keine messbare Wirkung auf Korrektheit
- **F-prompt-correctness.5** — Streuung bei Example-Mapping ist modellabhängig

Example-Mapping ist auf der neuartigen Kata der dominante Korrektheits-Hebel: +76 pp bei Opus 4.7, +64 pp bei Opus 4.6, +48 pp bei Sonnet 4.6 ohne Thinking. User-Story bleibt über alle Modelle innerhalb von 8 pp an Prosa — die Stakeholder-Perspektive liefert schlicht keine Information über die Domänenregeln. Der Hebel ist modell-gated: Haiku 4.5 erreicht in allen sechs Zellen 0.00. Überraschend wirkt Thinking bei Example-Mapping *negativ* und invers zur Modellstärke (Sonnet -36 pp, Opus 4.6 -15 pp, Opus 4.7 -2 pp); die Transcript-Analyse zeigt, dass schwächere Modelle die Beispiel-Semantik hinterfragen und Alternativ-Lesarten konstruieren. Caveat: eine Kata, ein Workflow. [findings.md](#)

1.2 RQ-prompt-known-kata — Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?

Datenbasis: 45 Runs · Coverage: 9/9 Zellen (100 %) bei min_replicates=5.

verification_pct nach Prompt-Stil × Modell (höher = besser; 🏆 = bester Stil pro Zeile, Ties alle):

Modell	prose	user-story	example-mapping
opus-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)
sonnet-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)

Modell	prose	user-story	example-mapping
haiku-4-5-portkey-no-thinking	0.24 ($\sigma=0.43$)	0.00 ($\sigma=0$)	0.63 🏆 ($\sigma=0.51$)

Befunde:

- **F-prompt-known-kata.1** — Opus und Sonnet liefern stilunabhängig perfekte Korrektheit
- **F-prompt-known-kata.2** — Haiku scheitert kapazitätsbedingt, nicht stilbedingt
- **F-prompt-known-kata.3** — H1 bestätigt: Prompt-Stil differenziert bei starken Modellen nicht
- **F-prompt-known-kata.4** — H4 bestätigt: Mehrdeutigkeits-Mechanismus greift nicht bei trainingsbekannter Kata
- **F-prompt-known-kata.5** — H2 kann nicht bewertet werden: Code-Qualität nur bei funktionierenden Runs vergleichbar
- **F-prompt-known-kata.6** — RQ-prompt-correctness-Prognose bestätigt: Prompt-Stil differenziert nicht auf trainingsbekannter Kata
- **F-prompt-known-kata.7** — Verification-Adapter eliminiert Interface-Artefakte

Die Gegenprobe zur vorigen RQ: Auf der trainingsbekannten Kata verschwindet der Prompt-Stil-Effekt für starke Modelle vollständig — Opus und Sonnet liefern 30/30 Runs bei `verification_pct` = 1.00, Spread zwischen den Stilen exakt 0 pp. Damit ist der Mechanismus geklärt: Example-Mapping wirkt über die Auflösung von Mehrdeutigkeiten, und die Conway-Regeln haben keine. Bei Haiku wirkt es über einen anderen Kanal — nicht Disambiguierung, sondern Aktivierung: In 5/5 user-story-Runs bricht das Modell sofort ab (12–17 s, kein Code), mit Beispielen arbeiten 4/5 Runs die Aufgabe durch. Methodisch wichtig: Der Verification-Adapter, der die Agent-Funktion direkt importiert statt über einen CLI-Vertrag zu prüfen, eliminierte Interface-Artefakte, die zuvor als Korrektheitsfehler erschienen. [findings.md](#)

2.1 RQ-model-quality — Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?

Datenbasis: 44 Runs · Coverage: 12/12 Zellen (100 %) bei `min_replicates=3`.

Code-Qualität nach Modell (Mittelwerte; kleiner = besser außer `verification_pct`; Quality-Pokale korrektkeits-gegated):

Modell	code_mass	smell_total	mccabe_max	cognitive_max	cc_longest_function	verification_pct	n
opus-5	172.67	1.67	3.00	2.00	6.33	1.00 🏆	3
opus-5-no-thinking	149.33	1.67 🏆	2.67	1.67	5.33	1.00 🏆	3
fable-5	163.00	3.00	2.00 🏆	1.00 🏆	8.33	1.00 🏆	3
fable-5-no-thinking	163.33	2.33	2.67	1.67	6.67	1.00 🏆	3
opus-4-8	145.33 🏆	2.67	4.33	5.33	4.33 🏆	1.00 🏆	3
opus-4-8-no-thinking	190.50	3.00	4.25	4.75	11.50	1.00 🏆	4
opus-4-7	159.00	2.33	3.33	3.00	7.00	1.00 🏆	3
opus-4-7-no-thinking	166.60	2.60	4.50	4.40	8.10	1.00 🏆	10
opus-4-6-portkey	173.00	4.33	6.67	12.00	19.33	1.00 🏆	3
opus-4-6-portkey-no-thinking	175.67	4.33	7.67	13.00	18.67	1.00 🏆	3
sonnet-4-6	178.00	5.67	6.33	11.00	21.67	1.00 🏆	3
sonnet-4-6-no-thinking	166.67	3.33	6.00	5.00	15.00	0.73	3

Befunde:

- **F-model-quality.1** — Korrektheit (innen + außen) auf v4 ist nahezu modellunabhängig perfekt
- **F-model-quality.2** — Modell-Ranking: Fable 5 und Opus 5 führen auf Komplexität, Opus 4.8 auf Code-Mass; alle drei deutlich vor Opus 4.6 und Sonnet
- **F-model-quality.3** — Thinking wirkt nicht uniform; bei Opus 4.8 stark auf Code-Größe, bei Opus 4.6 neutral, bei Sonnet negativ auf `cognitive_max`
- **F-model-quality.4** — Token-Kosten: Fable 5 und Sonnet/Opus 4.7 die günstigsten, Opus 4.8 der teuerste; Wallclock einheitlich
- **F-model-quality.5** — Vertrags-Konformität unter explizitem API-Vertrag fast vollständig erreicht; ein Sonnet-Ausreißer redefiniert `Cell` als Objekt

Bei gesättigter Korrektheit (elf von zwölf Zellen bei 1.00) trennt allein die Code-Qualität die Modelle — und zwar in drei komplementäre Profile statt eine Rangliste. Fable 5 und Opus 5 halten die Spitzen-Komplexität nahe am theoretischen Minimum (`cognitive_max` 1.0 bzw. 2.0), Opus 4.8 minimiert stattdessen Code-Mass (APP) und die längste Funktion (145.3 / 4.3). Der Abstand zur Vorgänger-Generation ist groß: auf `cognitive_max` trennt Fable 5 von Opus 4.6 ein Faktor ~12. Thinking wirkt nicht uniform — bei Opus 4.8 stark auf die Code-Größe (~45 `code_mass`), bei Sonnet dagegen klar schädlich (`cognitive_max` verdoppelt sich auf 11.0). Caveat: n=3 in den meisten Zellen, eine Kata, ein Workflow. [findings.md](#)

2.2 RQ-model-novel — Wie unterscheiden sich Fable 5, Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?

Datenbasis: 30 Runs · Coverage: 5/5 Zellen (100 %) bei `min_replicates=5`.

Korrektheit (außen) primär, Code-Qualität und Kosten sekundär (Klammerwerte = führend, aber ohne volle Korrektheits-Dekung):

Modell	n	verification_pct ↑	σ	cognitive_max ↓	mccabe_max ↓	smell_total ↓	total_tokens ↓	duration_s ↓
fable-5-no-thinking	5	0.83	0.10	(4.0)	(4.2)	(0.2)	13.4 M 🏆	7826
opus-5-no-thinking	5	0.88	0.11	2.8 🏆	3.8	0.2 🏆	24.6 M	5931
opus-4-8-no-thinking	5	0.92 🏆	0.09	7.4	7.0 🏆	1.2	31.0 M	5264
opus-4-7-no-thinking	10	0.67	0.36	10.5	7.9	1.8	13.7 M	3693 🏆
opus-4-6-portkey-no-thinking	5	0.93 🏆	0.08	22.2	10.6	5.6	15.1 M	4416

Befunde:

- **F-model-novel.1** — opus-4-8 und opus-4-6 lösen claim-office zuverlässig, opus-5 und fable-5 im Mittelfeld, opus-4-7 nicht
- **F-model-novel.2** — Workflow×Modell-Interaktion ist der dominierende Effekt
- **F-model-novel.3** — Korrektheit differenziert stärker als Code-Qualität
- **F-model-novel.4** — Präziserer Mechanismus auf opus-4-7: Test-Listen-Vollständigkeit, nicht Subagent-Isolation
- **F-model-novel.5** — opus-4-8 erkaufte beste Code-Qualität mit ~2× Token-Kosten
- **F-model-novel.6** — fable-5: sauberster, am meisten getesteter Code — aber nie die volle Spec

Auf der neuartigen Kata trennt nicht die Code-Qualität, sondern die Korrektheit selbst — und die naive Erwartung „neuer = besser“ trägt nicht: Vorne liegen die beiden Ränder der Opus-4er-Reihe (4.6 mit 0.93, 4.8 mit 0.92), während das mittlere 4.7 mit 0.67 bimodal abfällt. Der Mechanismus dahinter ist präzise lokalisiert: nicht die Subagent-Isolation, sondern eine unvollständige Test-Liste, die ganze Spec-Operationen ausblendet — eine einzige zusätzliche Pflicht („Cover every spec example“) hebt 4.7 auf 0.96 bei drastisch engerer Streuung. Fable 5 zeigt das Spiegelbild von Opus 4.8: der sauberste und meistgetestete Code der RQ, aber nie die volle Spec (max 14/15). Caveat: n=5, Routing-Unterschied zwischen 4.6 (Portkey) und den nativen Modellen. [findings.md](#)

3.1 RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?

Datenbasis: 49 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

verification_pct nach Workflow × Modell (höher = besser; 🏆 je Modell-Spalte — der Sieger wechselt modell-abhängig):

Workflow	opus-4-7 (n)	opus-4-6 (n)
v4-exact-subagents	0.67 (10)	0.93 (5) 🏆
v5-exact-single-context	0.87 (10)	0.87 (5)
v6-hybrid	1.00 (5) 🏆	0.68 (15)

Befunde:

- **F-workflow-model.1** — v4 und v6 tauschen je nach Modell die Plätze
- **F-workflow-model.2** — Mechanismus: Orchestrierungs-Delegation vs. expliziter Subagent-Prompt

Die kürzeste RQ mit der unbequemsten Aussage: Es gibt keinen universell besten Workflow. v4 und v6 tauschen je nach Modell die Plätze — v6 ist auf Opus 4.7 mit 1.00 optimal und auf 4.6 mit 0.68 instabil, v4 genau umgekehrt (0.93 / 0.67). Nur v5 ist modell-unabhängig konstant (0.87 in beiden Spalten). Der Mechanismus liegt in der Orchestrierungs-Verantwortung: v6 delegiert sie ans Modell (Skill-Invocation im geteilten Kontext), was 4.7 beherrscht, während 4.6 in etwa 40 % der Runs die halbe Spec verliert — es implementiert nur den Quote-Teil und ignoriert Claim vollständig, bei weiterhin grünen internen Tests. Konsequenz: Workflow-Empfehlungen ohne Modell-Kontext sind nicht generalisierbar. [findings.md](#)

4.1 RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualität aus, und macht die TDD-Striktheit einen Unterschied?

Datenbasis: 103 Runs · Coverage: 16/16 Zellen (100 %) bei min_replicates=5.

Code-Qualität pro Workflow (alle Metriken kleiner = besser; 🏆 = bester Wert pro Spalte. Nie über Katas gemittelt).

Kata game-of-life:

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	10	18.8	12.8	31.7	4.8	33.6	155.0
v2-iterative	10	16.2	11.6	32.1	4.1	32.5	157.8
v3-basic-tdd	10	21.8	13.7	32.5	6.0	31.9	165.6
v4.1-testlist-scope-fix	5	6.4 🏆	5.0 🏆	16.4	2.4 🏆	32.0	156.6
v5.1-testlist-scope-fix	5	17.6	10.2	20.8	4.8	26.6 🏆	154.0
v6.1-hybrid-...	10	6.5	5.2	14.2 🏆	2.4 🏆	29.2	153.7
v8a-delayed-refactor-agent	5	10.6	7.4	17.6	3.0	31.2	142.0 🏆

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v8b-delayed-refactor-native	5	9.0	6.8	17.6	2.4 🏆	31.0	145.8

Kata claim-office:

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	5	12.2	8.4	40.4	11.6	269.4	835.4
v2-iterative	5	11.4	8.4	41.4	15.8	268.6	851.0
v3-basic-tdd	5	19.8	15.4	51.6	16.8	317.4	992.4
v4.1-testlist-scope-fix	5	26.8 🚩	16.0 🚩	40.8	13.2	156.8 🏆	621.6 🏆
v5.1-testlist-scope-fix	6	14.8	10.2	32.7	6.8	167.2	692.7
v6.1-hybrid-...	7	5.7 🏆	5.7 🏆	18.1 🏆	1.3 🏆	191.1	861.3
v8a-delayed-refactor-agent	5	7.4	6.6	28.4	4.0	245.6	813.8
v8b-delayed-refactor-native	5	11.0	8.0	35.8	6.2	238.8	780.2

🚩 v4.1-claim-office ist bimodal (cognitive_max $\sigma=24$, max=68). Korrektheit: auf game-of-life alle acht Workflows bei verification_pct = 1.00; auf claim-office 0.28 (v1/v2) bis 1.00 (v3, v5.1, v6.1, v8a).

Befunde:

- **F-tdd-quality.1** — Strikte phasen-strukturierte Workflows mit Refactor-Phase senken die Komplexitäts-Spitzen drastisch
- **F-tdd-quality.2** — Naives “use TDD” (v3) bringt auf game-of-life keinen Komplexitäts-Vorteil gegenüber Non-TDD (v1/v2)
- **F-tdd-quality.3** — Single-Context (v5.1) verliert den Komplexitäts-Vorteil der phasen-isolierten Subagents (v4.1) — aber nur auf game-of-life
- **F-tdd-quality.4** — Korrektheit ist workflow-abhängig auf novel Kata; v1/v2 Vibe-Coding kollabiert auf claim-office
- **F-tdd-quality.5** — Kostenspanne zwischen Workflows ist eine Größenordnung; strikte Workflows sind 5–50× teurer; Kata-Komplexität skaliert linear
- **F-tdd-quality.6** — Vibe + End-Refactoring erreicht Volumen-Niveau der strikten TDD-Workflows zu Non-TDD-Kosten; Verzweigungs-Komplexität bleibt schwächer
- **F-tdd-quality.7** — Subagent-Mechanismus für End-Refactor schlägt Slash-Command auf großer Kata; gleichauf auf kleiner Kata
- **F-tdd-quality.8** — Test-Schreib-Phase rettet Korrektheit auf novel Kata; reines Vibe-Coding scheitert
- **F-tdd-quality.9** — v6.1-Hybrid ist der robusteste TDD-Workflow über beide Katas; v4.1 ist kata-instabil

Die tragende RQ des Snapshots, und sie trennt zwei Hebel, die gern als ein Paket verkauft werden. **Korrektheit** hängt allein an der Existenz einer Test-Phase: v1/v2 ohne Tests brechen auf claim-office auf 0.28 ein, jeder Workflow mit Test-Phase liegt bei ≥ 0.96 — auch die nachträglich testenden v8a/v8b.

Code-Qualität hängt dagegen ausschließlich am erzwungenen Refactor-Takt: v3 („use TDD“ ohne Struktur) produziert mit cognitive_max 19.8 den schlechtesten Code der Matrix, schlechter als Vibe-Coding (11.4), während v6.1 auf 5.7 kommt. Das TDD-Etikett allein trägt also nichts. Der Preis: v6.1 kostet 16× mehr Tokens als v8a für eine Verbesserung von 7.4 auf 5.7. Caveat: ein Modell (opus-4-7-no-thinking), n=5 pro claim-office-Zelle. [findings.md](#)

4.2 RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?

Datenbasis: 36 Runs · Coverage: 7/7 Zellen (100 %) bei min_replicates=3.

Korrektheit pro Workflow (höher = besser; 🏆 = bester Wert pro Spalte, Ties alle):

Workflow	n	verification_pct (mean $\pm$ std)	verification_passed / 15 (min – max)	tests_passing
v3-basic-tdd	5	1.00 $\pm$ 0 🏆	15 – 15	100 % 🏆
v4.1-testlist-scope-fix	5	0.96 $\pm$ 0.09	12 – 15	100 % 🏆
v5.1-testlist-scope-fix	6	1.00 $\pm$ 0 🏆	15 – 15	100 % 🏆
v6.1-hybrid-...	3	1.00 $\pm$ 0 🏆	15 – 15	100 % 🏆
v7.1-hybrid-green-refactor-...	3	0.98 $\pm$ 0.04	14 – 15	100 % 🏆

Befunde:

- **F-tdd-correctness.1** — Drei von fünf TDD-Workflows lösen claim-office perfekt; v4.1 und v7.1 verlieren vereinzelt Szenarien
- **F-tdd-correctness.2** — v4.1 erreicht Korrektheit nur über drastisch höheren Aufwand pro Zyklus
- **F-tdd-correctness.3** — Predictions-Rate-Vergleich ist verzerrt durch ungleiche Vorhersage-Basis
- **F-tdd-correctness.4** — Wallclock-Spanne ist 10×, Token-Spanne 9×; keine Korrektheits-Korrelation

Innerhalb der TDD-Familie ist Korrektheit kein knappes Gut mehr: Drei von fünf Varianten lösen claim-office in jedem Run vollständig, die beiden anderen verlieren je ein Szenario. Auffällig ist die Trennlinie — beide Workflows mit isoliertem Green-Subagent (v4.1, v7.1) tragen je einen Ausreißer, alle drei mit Green im geteilten Kontext sind perfekt; ein isolierter Green sieht die Test-Listen-Diskussion nicht und übersieht Edge-Cases. Der Aufwand dahinter spreizt dagegen um Faktor 9–10: v4.1 fährt 44.6 Zyklen und 54 Minuten pro Run für dieselbe Korrektheit, die v3 mit 3.8 Zyklen in 5 Minuten erreicht. Strukturierte Workflows rechtfertigen sich auf dieser Kata also nicht über Korrektheit, sondern über Code-Qualität. [findings.md](#)

4.3 RQ-context — Welche Form der Kontext-Strukturierung — isolierte Subagent-Kontexte pro TDD-Phase (v4.1), ein geteilter, akkumulierter Single-Context (v5.1), ein Hybrid mit Skill-basiertem Red/Green im Shared-Context und isoliertem Refactor-Subagent (v6.1) oder ein Hybrid mit isolierten Green- und Refactor-Subagents bei Shared-Context-Test-Liste/Red (v7.1) — führt zu besserer Code-Qualität?

Datenbasis: 21 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=3.

Code-Qualität, Korrektheit, Kosten nach Kontext-Architektur 🏆 = bester Wert pro Spalte; verification_pct höher = besser, alle übrigen kleiner = besser):

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	code_mass	cc_loc	verification_pct	duration_seconds	total_tokens
v4.1 (alle isoliert)	5	26.8 ± 24.1	16.0 ± 9.0	40.8 ± 27.1	13.2 ± 7.5	621.6 ± 65.6 🏆	156.8 ± 38.0 🏆	0.96 ± 0.09	3 229 ± 920	14.10 M ± 2.99 🏆
v5.1 (alle geteilt)	6	14.8 ± 4.2	10.2 ± 2.6	32.7 ± 10.2	6.8 ± 7.6	692.7 ± 78.8	167.2 ± 27.9	1.00 ± 0 🏆	641 ± 122 🏆	18.73 M ± 5.35
v6.1 (Refactor isoliert)	3	4.3 ± 1.5 🏆	5.0 ± 1.7 🏆	16.7 ± 6.7 🏆	1.3 ± 1.2 🏆	920.7 ± 55.2	184.3 ± 4.9	1.00 ± 0 🏆	1 424 ± 781	30.16 M ± 18.56
v7.1 (Green + Refactor isoliert)	3	5.0 ± 1.0 🏆	4.67 ± 0.58 🏆	19.3 ± 2.5 🏆	2.3 ± 2.3 🏆	801 ± 3.6	187.3 ± 29.2	0.98 ± 0.04	1 970 ± 715	26.11 M ± 6.20

Befunde:

- **F-context.1** — Refactor-Subagent liefert den Komplexitäts-Vorteil; zusätzliche Green-Isolation ändert das Bild nicht
- **F-context.2** — Refactor-Subagent verteilt Funktionalität auf mehr Bausteine; Green-Isolation bremst den Mehr-Code-Effekt
- **F-context.3** — Korrektheit unterscheidet die Architekturen nicht
- **F-context.4** — Vier sehr unterschiedliche Kosten-Profile
- **F-context.5** — Zwei Hybrid-Positionen mit ähnlicher Code-Qualität, unterschiedlichem Kosten-Profil

Der Vier-Punkt-Vergleich lokalisiert den Qualitäts-Hebel präzise: Er entsteht aus dem **isolierten Refactor-Subagent** und aus nichts sonst. v6.1 und v7.1 teilen genau dieses Element und erreichen praktisch identische Komplexitäts-Spitzen (cognitive_max 4.3 / 5.0, alle Differenzen innerhalb σ); die zusätzliche Green-Isolation in v7.1 bringt keinen weiteren Gewinn. Wenn dagegen *alle* Phasen isoliert laufen (v4.1), schadet es — die Subagents konstruieren die Gesamt-Architektur über 44.6 Zyklen immer wieder neu und akkumulieren Komplexität, die keine Phase als Ganzes sieht (σ cognitive_max = 24.1). Bemerkenswert: v6.1 schreibt mit 920 LOC am meisten Code und hat trotzdem die wenigsten Smells — Sauberkeit kommt aus der Struktur, nicht aus Sparsamkeit. Caveat: n=3 in den Hybrid-Zellen. [findings.md](#)

4.4 RQ-pocock-vs-v62 — Wie schneidet der externe Matt-Pocock-TDD-Skill (v9-pocock-tdd: Single-Skill, Inline-Phasen, Tail-Refactor) auf claim-office-example-mapping gegen die interne Default-Baseline v6.2-with-why-cleaned (Multi-Command + Refactor-Subagent, Per-Cycle-Refactor) ab — auf Korrektheit, Code-Qualität, TDD-Disziplin und Kosten?

Datenbasis: 11 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=3.

Externer Single-Skill-TDD gegen die interne Multi-Command-Baseline:

Achse	v6.2-with-why-cleaned (n=8)	v9-pocock-tdd (n=3)	Sieger
Korrektheit verification_pct (höher = besser)	0.96 ± 0.09	1.00 ± 0 🏆	Pocock leicht
tests_passing rate	100 %	100 %	Tie 🏆🏆
Code-Qualität cognitive_max (kleiner = besser)	5.00 ± 1.77 🏆	14.33 ± 1.53	v6.2
mccabe_max (kleiner = besser)	4.50 ± 0.76 🏆	11.67 ± 0.58	v6.2
cc_longest_function (kleiner = besser)	12.38 ± 1.41 🏆	32.33 ± 1.53	v6.2
smell_total (kleiner = besser)	0.38 ± 0.74 🏆	6.67 ± 8.96	v6.2
code_mass (kleiner = besser)	878.5 ± 91	748.3 ± 62 🏆	Pocock
Kosten duration_seconds (kleiner = besser)	2530 ± 401	570 ± 106 🏆	Pocock
total_tokens (kleiner = besser)	44.4 M ± 3.4 M	13.1 M ± 4.6 M 🏆	Pocock
Disziplin refactorings_applied	24.88 ± 6.90	0 ± 0	by-design unterschiedlich
cycle_count	37.38 ± 1.60	14.00 ± 3.46	by-design unterschiedlich
tests_passed_immediately (kleiner = strikter)	15.12 ± 5.84	2.33 ± 4.04 🏆	Pocock
predictions_correct_rate (höher = besser)	97.2 % 🏆	89.9 %	v6.2

Befunde:

- **F-4.4.1** — Pocock und v6.2 gleichwertig korrekt
- **F-4.4.2** — v6.2 produziert saubereren Code, Pocock kompakteren

- **F-4.4.3** — Pocock ~70–78 % günstiger
- **F-4.4.4** — Tail-Refactor löst auf claim-office nicht aus
- **F-4.4.5** — Pocock macht weniger, größere Schritte
- **F-4.4.6** — Pocock skippt seltener

Ein extern entstandener TDD-Skill mit Inline-Phasen und Tail-Refactor erreicht dieselbe Korrektheit wie die interne Baseline (1.00 vs 0.96) bei 70–78 % geringeren Kosten — und liefert dabei um Faktoren schlechteren Code (`cognitive_max` 14.3 vs 5.0, `smell_total` 6.7 vs 0.4). Die Ursache ist sauber identifiziert und stützt den Refactor-Takt-Befund aus §4.6: Die Tail-Formulierung „After all tests pass, look for refactor candidates“ löste in 3/3 Runs **null** Refactorings aus, während die Per-Cycle-Variante 24.9 fährt. Bei grünen Tests stuft das Modell den Code ohne zusätzlichen Prompt-Druck als gut genug ein. Caveat: n=3 auf der Pocock-Seite — die Effektgrößen sind mit > 3 σ aber richtungsstabil. [findings.md](#)

5.1 RQ-stability — Wie stabil sind Code-Qualitaet und TDD-Disziplin pro Workflow ueber Replikate, und unter welchen Bedingungen ist n=3 als Replikat-Anzahl ausreichend?

Datenbasis: 59 Runs · Coverage: 5/6 Zellen (83 %) bei `min_replicates=10`.

Code-Qualität nach Workflow bei n=10 (kleiner = besser; 🏆 = bester Wert pro Spalte):

Workflow	<code>code_mass</code>	<code>smell_total</code>	<code>mccabe_max</code>	<code>cognitive_max</code>	<code>cc_longest_function</code>	n
v1-oneshot (prose)	155.00	4.80	12.80	18.80	31.70	10
v2-iterative (prose)	157.80	4.10	11.60	16.20	32.10	10
v3-basic-tdd (EM)	165.60	6.00	13.70	21.80	32.50	10
v4-exact-subagents (EM)	166.60	2.60	4.50 🏆	4.40 🏆	8.10 🏆	10
v5-exact-single-context (EM)	152.60 🏆	4.10	8.90	14.50	17.40	10
v6-hybrid (EM)	158.60	2.20 🏆	4.50 🏆	5.20	13.10	10

Befunde:

- **F-stability.1** — Der Kernbefund aus RQ-tdd-quality (v4 dominiert Code-Komplexität, v3 ist Schlusslicht) repliziert bei n=10 mit gleichem Vorzeichen
- **F-stability.2** — Workflow-Stabilität ist nicht uniform; v4 hat 10 %-Outlier-Rate trotz tiefem typischen Wert; v5 ist das breiteste Workflow
- **F-stability.3** — Bei n=3 ist die volle Workflow-Rangordnung nur in ~25–60 % der Fälle korrekt; v4 als “Bester” ist robuster
- **F-stability.4** — Korrektheit bleibt bei n=10 modell-/workflow-unabhängig 100 %
- **F-stability.5** — Token-Verbrauch zeigt extrem hohe Streuung bei v4 und v5
- **F-stability.6** — TDD-Disziplin bildet workflow-charakteristische Banden
- **F-stability.7** — Test-Stärke (`mutation_score`) hat ein eigenes Stabilitätsprofil; v6-hybrid ist der stabilste Workflow, v4 der instabilste

Die methodische Absicherung des Labs: Bei n=10 hält die Rangordnung an den Rändern, aber nicht in der Mitte — „v4 ist deutlich besser als alles andere“ ist mit n=3 zuverlässig erkennbar, die vollständige Fünf-Workflow-Ordnung dagegen nur in 16–63 % der Subsamples. Daraus folgt die Lab-Regel: Große Effekte dürfen mit n=3 berichtet werden, marginale Unterschiede brauchen $n \geq 10$. Wichtig ist die Stabilitäts-Asymmetrie: v4 hat den besten Median, entgleist aber in 1 von 10 Runs (`cognitive_max` = 17, gesamte Logik in einer 28-Zeilen-Funktion), während v6 mit 0 % Outlier-Rate und $\sigma = 0.005$ beim `mutation_score` durchgängig planbar bleibt. Der Mittelwert allein verdeckt das. Caveat: ein Modell, eine Kata. [findings.md](#)

Forschungsfragen (OpenCode)

1.1 RQ-model-quality-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle (Opus 4.7 via Portkey + vier Nicht-Anthropic-Modelle aus dem Portkey-Catalog) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?

Datenbasis: 30 Runs · Coverage: 6/6 Zellen (100 %) bei `min_replicates=5`.

Code-Qualität als primärer Outcome, Korrektheit als Gating-Voraussetzung (Auszug; Pokale nur für Modelle mit `verification_pct = 1.0`):

Metrik	Richtung	opus-4-7-portkey	glm-5-1	gemin-3-5-flash	kimi-k2-6	deepseek-v4-flash	deepseek-v4-pro
<code>verification_pct</code> (mean)	höher	1.00 🏆	1.00 🏆	1.00 🏆	0.57	1.00 🏆	0.85
<code>smell_total</code> (mean)	kleiner	3.6	2.8 🏆	4.0	4.4	4.0	4.2
<code>cognitive_max</code> (mean)	kleiner	11.4 🏆	11.6 🏆	16.0	9.4	13.2	11.4
<code>mccabe_max</code> (mean)	kleiner	7.6	7.0 🏆	10.4	7.6	9.4	8.6
<code>cc_longest_function</code> (mean)	kleiner	18.6 🏆	19.8	18.6 🏆	15.2	27.6	15.0
<code>lines_of_code</code> (mean)	kleiner	38.2 🏆	46.4	52.2	22.4	44.8	24.6
<code>duration_seconds</code> (mean)	kleiner	231	835	153 🏆	1083	612	2381
<code>cost_usd</code> (mean, \$/perfect-run)	kleiner	\$1.84	\$0.74	\$1.06	\$2.65	\$0.10 🏆	\$0.46

Befunde:

- **F-1.1** — Opus 4.7 schreibt die kompakteste Implementierung

- **F-1.2** — GLM 5.1 hält Opus-Niveau in Komplexität
- **F-1.3** — Kimi-K2 schreibt zu wenige Tests, scheitert an externer Verifikation
- **F-1.4** — Gemini 3.5 Flash: schnell, aber komplexester Code
- **F-1.5** — Skill-Tool-Compliance ist modellabhängig
- **F-1.6** — DeepSeek-V4-Flash: günstigster Pfad zur korrekten Lösung
- **F-1.7** — DeepSeek-V4-Pro: Skill-Compliance-Champion, aber Tail-Risk in Duration

Über einen zweiten Harness werden Nicht-Anthropic-Modelle vergleichbar — und das Preis-Qualitäts-Feld fächert weit auf. Opus 4.7 schreibt die kompakteste Lösung (38.2 Produktiv-LoC, Median 3.3 LoC pro Funktion — es extrahiert konsequent kleine Helfer), GLM 5.1 hält bei der Komplexität Opus-Niveau zu einem Drittel der Kosten, und DeepSeek-V4-Flash erreicht volle Korrektheit für ~\$0.10 pro Run, also eine Größenordnung unter Opus' \$1.84. Der wichtigste methodische Befund betrifft die Marker-Compliance: Gemini Flash schreibt praktisch keine Prediction-Marker (0.4 pro Run) und ist trotzdem voll korrekt — niedrige Marker-Zahlen messen Format-Erkennung, nicht TDD-Disziplin. Kimi-K2 fällt aus dem Trophy-Pool (0.57), weil es die Test-Liste minimiert. Caveat: n=5, eine Kata. [findings.md](#)

1.2 RQ-model-novel-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?

Datenbasis: 40 Runs · Coverage: 8/8 Zellen (100 %) bei min_replicates=5.

Korrektheit (außen) primär, Code-Qualität sekundär (Auszug; Pokale nur bei verification_pct = 1.0):

Metrik	Richtung	opus-4-7-portkey	glm-5-1	mistral-medium-3-5	kimi-k2-6	gemini-3-5-flash	deepseek-v4-flash	deepseek-v4-pro	minimax-m2-7
verification_pct (mean)	höher	1.00 🏆	1.00 🏆	0.95	0.84	0.80	0.60	0.60	0.04
smell_total (mean)	kleiner	0.8 🏆	4.0	23.6	20	18	13.4	16.6	10.2
cognitive_max (mean)	kleiner	9.8 🏆	12.2	74.8	21.8	40.2	11.6	17.4	11.4
mccabe_max (mean)	kleiner	7.6 🏆	9.2	33.6	17.6	23.4	9.2	11.0	7.6
cc_longest_function (mean)	kleiner	25.4 🏆	28.8	120	54.4	98.4	31.6	42.2	30.0
cost_usd (mean, \$/Run)	kleiner	\$5.90	\$2.10 🏆	\$24.69 †	\$2.78	\$2.23	\$0.28 ‡	\$0.11 ‡	\$2.40
duration_seconds (mean)	kleiner	664 🏆	1726	4051	1811	395	1279	956	1428

† Mistral-Kosten sind ein OpenCode-Integrations-Artefakt (fehlendes Prompt-Caching), kein Modell-Befund. ‡ DeepSeek-Werte inklusive zweier CLI-Vertrags-Abbrüche.

Befunde:

- **F-1.1** — Opus 4.7 und GLM 5.1 erreichen vollständige Korrektheit; Tradeoff Code-Qualität ↔ Kosten
- **F-1.2** — Kimi K2.6 und Gemini 3.5 Flash: Spitzen-Korrektheit mit Varianz-Tail
- **F-1.3** — MiniMax M2.7: stabiles Spec-Misverständnis, kein Einzelfall
- **F-1.4** — Predictions-Format-Compliance ist NICHT prädiktiv für Korrektheit
- **F-1.5** — Code-Mass-Spread innerhalb Modell: Flash und MiniMax bimodal/breit
- **F-1.6** — Cost-Effizienz pro perfektem Lauf: GLM 5.1 deterministisch UND günstig
- **F-1.7** — Mistral Medium 3.5: hohe Korrektheit gegen hohe Komplexität und höchste Kosten
- **F-1.8** — DeepSeek V4 (flash + pro): Workflow-Compat-Drop dominiert über Spec-Verstehen

Auf der neuartigen Kata trennt sich das Modell-Feld drastisch: Opus 4.7 und GLM 5.1 lösen alle fünf Replikate vollständig, MiniMax M2.7 scheitert in fünf von fünf (0.04) — und zwar reproduzierbar, mit 30.8 selbst geschriebenen, grünen Tests. Das Modell baut konsistent eine andere Spec als die Verifikations-Suite erwartet; interne Tests belegen hier gar nichts. GLM 5.1 ist der praktische Sieger auf der Kosten-Achse: dieselbe Determinismus-Garantie wie Opus zu einem Drittel des Preises (\$2.10 vs \$5.90), erkaufte mit etwas höherer Komplexität. Zwei Caveats trüben Einzelwerte: Mistrals extreme Kosten sind ein Caching-Integrationsfehler des Harness, und die DeepSeek-Werte mischen zwei Fehlermodi (CLI-Vertragsbruch in frühen Runs vs. Modell-Leistung). [findings.md](#)

Forschungsfragen (pi)

1.1 RQ-model-quality-pi — Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v6.2.1-phase-continuation-pi-Workflow?

Datenbasis: 50 Runs · Coverage: 10/10 Zellen (100 %) bei min_replicates=5.

Code-Qualität, kleiner = besser (Pokale nur für Zellen mit tests_passing = 100 %):

Modell	smell_total	cognitive_max	mccabe_max	code_mass	cc_longest_function	tests_passing
glm-5-2	1.0 🏆	7.8	6.6	178.2	22.6	100 %
sonnet-5	2.2	6.6 🏆	5.0 🏆	183.0	19.6	100 %
kimi-k2-7	3.0	10.8	7.2	150.4	21.6	100 %
opus-4-8	3.4	9.6	6.8	149.2	17.4	100 %
gpt-5-6-sol	3.6	13.4	9.4	134.8 🏆	21.2	100 %

Modell	smell_total	cognitive_max	mccabe_max	code_mass	cc_longest_function	tests_passing
deepseek-v4-pro	4.0	14.0	10.2	158.4	25.4	100 %
minimax-m3	8.4	6.6 🏆	5.2	212.2	15.0 🏆	100 %
glm-5-1	2.2	7.2	6.0	144.8	22.2	80 %
gpt-5-6-terra	6.0	7.8	6.0	136.4	23.2	80 %
qwen3-235b	1.8	6.4	3.4	206.6	42.4	0 %

Befunde:

- **F-1.1** — glm-5-2 liefert den saubersten Code, sonnet die niedrigste Komplexität
- **F-1.2** — deepseek und gpt-5-6-sol lösen die Kata korrekt, aber mit hoher Komplexität
- **F-1.3** — Korrektheit clustert oben, mit qwen als Total-Fail
- **F-1.4** — TDD-Disziplin variiert stark ohne Korrektheits-Korrelation
- **F-1.5** — Kosten spreizen um das 6-Fache bei vergleichbarer Qualität

Der dritte Harness bringt das breiteste Modell-Feld (zehn Zellen) und zeigt, dass die Qualitäts-Achsen teilweise orthogonal laufen: glm-5-2 führt bei der Smell-Summe (1.0), sonnet-5 bei beiden Komplexitätsmaßen (6.6 / 5.0) — kein Modell dominiert alle drei. Korrektheit clustert oben (sieben von zehn Zellen bei `verification_pct` = 1.00), mit qwen3-235b als klarem Boden: Es baut Code, bekommt ihn aber in keinem Run grün — ein echtes Kompetenz-Defizit, kein Abbruch. Praktisch relevant ist die Kosten-Lücke: Die smell-ärmsten Modelle sind zugleich die teuersten (glm-5-2 ~\$2.53, sonnet-5 ~\$2.83 gegen kimi-k2-7 ~\$0.60); ein „billig UND sauber“-Modell fehlt im Feld. Caveat: Kosten sind Listenpreis-Schätzung, kein abgerechneter Betrag. [findings.md](#)

1.2 RQ-model-novel-pi — Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v6.2-with-why-cleaned-pi-Workflow?

Datenbasis: 75 Runs · Coverage: 15/15 Zellen (100 %) bei `min_replicates`=5.

Korrektheit (außen), höher = besser 🏆 nur für Zellen mit `verification_pct` ≥ 0.99 bei $\sigma \leq 0.03$):

Modell	verification_pct mean	σ	tests_passing-Rate
opus-4-8-no-thinking	1.00 🏆	0.00	100 %
glm-5-2	1.00 🏆	0.00	100 %
gpt-5-6-sol	1.00 🏆	0.00	100 %
kimi-k2-7	1.00 🏆	0.00	100 %
opus-4-8	0.99 🏆	0.03	100 %
sonnet-5-no-thinking	0.84	0.15	100 %
deepseek-v4-pro-no-thinking	0.80	0.45	80 %
minimax-m3-no-thinking	0.77	0.44	80 %
kimi-k2-7-no-thinking	0.73	0.42	80 %
sonnet-5	0.72	0.19	100 %
gpt-5-6-terra	0.69	0.42	80 %
deepseek-v4-pro	0.60	0.55	100 %
minimax-m3	0.20	0.45	100 %
qwen3-235b	0.00	0.00	0 %
qwen3-235b-no-thinking	0.00	0.00	0 %

Befunde:

- **F-1.1** — Korrektheit clustert dichotom, mit gradueller Mittelzone
- **F-1.2** — qwen3-235b baut Code, löst die Kata aber nie
- **F-1.3** — TDD-Disziplin und Korrektheit korrelieren nicht
- **F-1.4** — Der Reasoning-Schalter verschiebt die Korrektheit nicht
- **F-1.5** — Perfekte Korrektheit bei stark unterschiedlichen Kosten

Mit 15 Zellen die breiteste Modell-Erhebung des Labs, und sie zeigt eine dreiteilige Verteilung: ein Perfekt-Cluster (fünf Zellen bei ≈ 1.00 , $\sigma \leq 0.03$), ein Total-Fail-Cluster (qwen3-235b in beiden Armen bei 0.00) und eine breite, run-instabile Mitte (0.20–0.84 mit σ bis 0.55) — dieselbe Zelle schwankt dort über die Replikate zwischen 0 und 15 bestandenen Szenarien. Zwei Befunde mit Praxis-Wert: Der Reasoning-Schalter verschiebt die Korrektheit nicht (selbst bei Opus 4.8, wo er nachweislich greift, ± 0.01), und die Marker-Compliance sagt nichts über das Ergebnis — gpt-5-6-sol löst die Kata mit 20.8 Predictions ebenso perfekt wie Opus mit 70. Bei den Kosten spreizen die perfekten Zellen um Faktor 5.7 (\$2.54 bis \$14.43). [findings.md](#)

Forschungsfragen (Cursor CLI)

1.1 RQ-model-quality-cursor — Wie unterscheiden sich die via cursor-cli-Harness erreichbaren Modelle (Opus, Composer, Grok) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping?

Datenbasis: 15 Runs · Coverage: 3/3 Zellen (100 %) bei `min_replicates`=5.

Code-Qualität und Korrektheit (alle Qualitätsmetriken kleiner = besser; alle drei Zellen 100 % korrekt, daher kein Korrektheits-Gating):

Metrik (Richtung)	opus-cursor	composer-cursor	grok-cursor
cognitive_max (↓)	16.6	8.2 🏆	10.6
cognitive_avg (↓)	15.3	5.93 🏆	7.2
mccabe_max (↓)	10.6	7.6 🏆	8.8
mccabe_avg (↓)	4.33	2.63 🏆	3.38
Smell-Summe smell_total (↓)	4.0	3.4 🏆	3.6
Produktiv-LoC lines_of_code (↓)	27.8 🏆	59.2	42.8
Code-Mass (APP) code_mass (↓)	141.8 🏆	182.2	149.2
Korrektheit (innen) tests_passing (↑)	100 % 🏆	100 % 🏆	100 % 🏆
duration_seconds (↓)	198	120.8 🏆	169
total_tokens (↓)	1.74 M	768 k 🏆	1.17 M

Befunde:

- **F-1.1** — Composer schreibt die komplexitätsärmste, Opus die knappste Lösung
- **F-1.2** — Modell-Spreizung bestätigt: der cursor-cli-Harness ist diskriminationsfähig
- **F-1.3** — Opus nutzt die TDD-Marker-Mechanik am dichtesten

Der jüngste Harness im Lab, und er liefert den saubersten Beleg dafür, dass **Knappheit und Einfachheit auseinanderfallen**: Opus schreibt mit 27.8 Produktiv-LoC die kürzeste Lösung und trägt zugleich die höchste kognitive Komplexität (16.6), Composer schreibt mit 59.2 LoC mehr als doppelt so viel Code bei halber Komplexität (8.2). Die Code-Inspektion bestätigt den Mechanismus: Opus packt die Logik in eine dichte Funktion mit dreifach verschachtelten Schleifen, Composer extrahiert benannte Konstanten und Helfer und trennt die Durchläufe in flache Einzelschritte. Wer „weniger Code“ als Qualitätsproxy nutzt, misst hier das Gegenteil von Wartbarkeit. Caveat: Kosten laufen über das Cursor-Abo, daher kein Kostenvergleich mit den anderen Harnessen. [findings.md](#)

Forschungsfragen (Harness-übergreifend)

1.1 RQ-harness — Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität und TDD-Disziplin aus, wenn Modell, Workflow-Intention und Prompt-Stil konstant gehalten werden?

Datenbasis: 38 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Pivot über sechs Zellen (Kata × Harness) bei konstantem Modell, Workflow und Prompt-Stil:

Outcome	Richtung	CC × claim (n=8)	OC × claim (n=5)	pi × claim (n=5)	CC × GOL (n=10)	OC × GOL (n=5)	pi × GOL (n=5)
verification_pct (mean ± σ)	höher = besser	0.96 ± 0.09	1.00 🏆 ± 0	1.00 🏆 ± 0	1.00 🏆 ± 0	1.00 🏆 ± 0	1.00 🏆 ± 0
input+output (mean, k Tokens, ohne cache)	kleiner = besser	161 🏆	468	1971	36 🏆	156	287
cost_usd (mean, Listpreis)	kleiner = besser	\$30.47	\$18.80	\$11.20 🏆	\$6.22	\$2.26	\$1.65 🏆
duration_seconds (mean)	kleiner = besser	2530 ± 401	2230 ± 952	1647 🏆 ± 205	627 ± 117	516 ± 196	317 🏆 ± 43
code_mass (APP, mean)	kleiner = besser	879 ± 91	827 ± 99	807 🏆 ± 16	153 ± 14	149 🏆 ± 12	158 ± 13
cognitive_max (mean)	kleiner = besser	5.0 ± 1.8	4.8 ± 3.0	4.2 🏆 ± 1.6	4.3 🏆 ± 2.8	6.2 ± 2.6	7.6 ± 3.1
cc_longest_function (mean)	kleiner = besser	12.4 🏆 ± 1.4	15.0 ± 7.0	14.6 ± 1.7	12.2 🏆 ± 6.9	17.0 ± 5.2	18.2 ± 5.3
refactorings_applied (mean)	höher = besser	24.9 🏆 ± 6.9	19.0 ± 11.4	16.8 ± 2.8	7.9 🏆 ± 1.9	5.0 ± 2.8	3.0 ± 0.7

total_tokens ist über die Harnesse nicht direkt vergleichbar (unterschiedliche Cache-Zählung) — der faire Effizienz-Proxy ist input + output .

Befunde:

- **F-harness.1** — Korrektheit ist harness-invariant; CC × claim-office zeigt leichte Streuung
- **F-harness.2** — Token-Footprint und Listpreis-Kosten: pi ist die günstigste Variante
- **F-harness.3** — Code-Mass (APP) ist harness-invariant; mccabe/longest/cognitive variieren kata-abhängig
- **F-harness.4** — Claude-Code-Harness-Glitch: premature end_turn auf claim-office (Thinking-Variante)
- **F-harness.5** — TDD-Disziplin ist harness-invariant; Refactor-Häufigkeit fällt CC → OC → pi monoton ab
- **F-harness.6** — Pi-Cycle-Inflation auf claim-office: deutlich mehr Red-Marker als CC/OC bei gleicher Test-Anzahl

Bei konstantem Modell und Workflow ist die Korrektheit harness-invariant (fünf von sechs Zellen deterministisch perfekt) — die Wahl des Agent-Werkzeugs verschiebt also Kosten und Qualitätsprofil, nicht die Ergebnislänge. Am lehrreichsten ist der scheinbare Widerspruch bei den Kosten: pi jagt auf claim-office etwa das Zwölfwache an frischen Input-Tokens durch das Modell wie Claude Code (1971 k vs 161 k) und ist trotzdem knapp ein Drittel so teuer. Auflösung: CC hält die Roh-Token-Zahl niedrig, indem es denselben wachsenden Kontext über ~37 Zyklen immer wieder durch den Cache schickt — 44 Millionen Cache-Reads, die selbst zum Rabatttarif teurer sind als pi's cache-lose Vollpreisrechnung. Caveat: eingefrorener Portkey-Snapshot; die Kostenfrage wird in \$4.6.2 unter Requesty neu gemessen. [findings.md](#)

1.2 RQ-harness-requesty — Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität, TDD-Disziplin und Kosten aus, wenn Modell (opus-4-8 über Requesty), Workflow-Intention und Prompt-Stil konstant gehalten werden?

Datenbasis: 40 Runs · Coverage: 8/8 Zellen (100 %) bei min_replicates=5.

Vier Harnesse bei konstantem Modell (opus-4-8 über Requesty), Workflow und Prompt-Stil.

Kata claim-office (Korrektheit außen zählt):

Metrik (Richtung)	CC	OC	pi	cursor
verification_pct (höher)	0.93	0.88	0.99	1.0 🏆
tests_passing-Rate (höher)	100 % 🏆	100 % 🏆	100 % 🏆	100 % 🏆
cost_usd \$ (kleiner)	32.89	22.30	14.43	9.22 🏆
total_tokens (kleiner)	49.9 M	34.1 M	13.8 M	13.8 M 🏆
duration_seconds (kleiner)	3149	2393	1884	1001 🏆

Kata game-of-life (alle Zellen verification_pct = 1.0):

Metrik (Richtung)	CC	OC	pi	cursor
cost_usd \$ (kleiner)	3.45	1.99	1.78	1.48 🏆
total_tokens (kleiner)	4.09 M	1.96 M	1.07 M 🏆	1.74 M
cognitive_max (kleiner)	5.0 🏆	12.6	11.0	16.6
mccabe_max (kleiner)	4.6 🏆	8.8	8.0	10.6
smell_total (kleiner)	2.2 🏆	3.2	3.4	4.0
refactorings_applied (höher)	8.8 🏆	3.2	2.8	2.6

Cursor läuft auf claude-opus-4-8-medium (medium effort) — der Qualitäts-Rückstand ist Effort- und Harness-Effekt zugleich und nicht trennbar.

Befunde:

- **F-1.1** — Korrektheit ist harness-invariant
- **F-1.2** — cursor ist der günstigste und schnellste Harness; pi führt unter CC/OC/pi
- **F-1.3** — Claude Code liefert auf game-of-life die schlankste Spitzen-Komplexität; cursor die höchste
- **F-1.4** — TDD-Disziplin ist über alle Harnesse strukturgleich, außer Refactor-Intensität

Die Wiederholung des Harness-Vergleichs unter einheitlicher Kostenmessung und mit cursor als viertem Arm bestätigt den Kernbefund: Korrektheit ist harness-invariant (0.88–1.00, alle Differenzen innerhalb der Replikat-Streuung), Kosten spreizen um Faktor 3.5 (\$9.22 bis \$32.89) und Wallclock um Faktor 3. Der Qualitäts-Unterschied auf game-of-life ist deutlich und mechanistisch erklärt: Claude Code hält cognitive_max bei 5.0 gegen 11.0–16.6 der anderen — und wendet dafür mit 8.8 fast dreimal so viele Refactorings an. Cursor gewinnt die Kosten-Achse teilweise über den Tarif (nativer Listpreis statt Requesty-Aufschlag), nicht allein über Effizienz. Caveat: cursor läuft auf einem medium-effort-Modell, sein Qualitäts-Rückstand ist konfundiert. [findings.md](#)

1.3 RQ-cost-sol-pi-vs-opus-cc — Wie viel günstiger ist das GPT-Modell gpt-5-6-sol auf dem pi-Harness gegenüber opus-4-8 auf Claude Code — bei gleichem Prompt-Stil und outcome-äquivalentem TDD-Workflow, über beide Katas?

Datenbasis: 20 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=5.

Kosten-Umstiegs-Vergleich zweier Praxis-Bündel (Modell UND Harness variieren gemeinsam):

Kata claim-office:

Metrik (Richtung)	sol-pi	opus-cc
cost_usd \$ (kleiner)	2.54 🏆	32.89
total_tokens (kleiner)	2.09 M 🏆	49.9 M
duration_seconds (kleiner)	503 🏆	3149
verification_pct (höher)	1.00 🏆	0.93
cognitive_max (kleiner)	9.2	3.0 🏆
mccabe_max (kleiner)	6.8	3.8 🏆
Smell-Summe smell_total (kleiner)	15.4	0.0 🏆

Kata game-of-life:

Metrik (Richtung)	sol-pi	opus-cc
cost_usd \$ (kleiner)	1.09 🏆	3.45
duration_seconds (kleiner)	240 🏆	719
verification_pct (höher)	1.0 🏆	1.0 🏆
cognitive_max (kleiner)	13.4	5.0 🏆
Smell-Summe smell_total (kleiner)	3.6	2.2 🏆

Befunde:

- **F-1.1** — sol-pi ist auf beiden Katas drastisch günstiger — auf der teuren Kata ~13×
- **F-1.2** — Der Preisvorteil kostet keine Korrektheit — auf claim-office ist sol-pi sogar genauer
- **F-1.3** — Billiger heißt nicht sauberer: sol-pi trägt durchweg höhere Komplexität und mehr Smells

Die Umstiegs-Frage in Reinform: Wer vom teuren auf das günstige Bündel wechselt, spart je nach Kata 68 % bis 92 % und läuft rund dreimal schneller — **ohne Korrektheit einzubüßen**; auf der neuartigen Kata ist das günstige Bündel mit 1.00 ($\sigma = 0$) sogar konsistenter als das teure (0.93, $\sigma = 0.12$). Der Preis ist Wartbarkeit, und zwar deutlich: auf claim-office steht eine Smell-Summe von 15.4 gegen 0.0 und `cognitive_max` 9.2 gegen 3.0. Faustregel: das günstige Bündel für durchsatzkritische Arbeit mit tolerierbarer Nachbearbeitung, das teure dort, wo niedrige Komplexität den Aufpreis rechtfertigt. Bindender Caveat: Modell und Harness variieren gemeinsam — der Effekt ist ihre Summe, nicht einer allein. [findings.md](#)

1.4 RQ-model-quality-cc-vs-pi — Unterscheidet sich das Code-Qualitäts-Profil von Opus (opus-4-8) zwischen dem Claude-Code- und dem pi-Harness, je mit und ohne Thinking, bei konstanter Workflow-Generation (v6.2)?

Datenbasis: 20 Runs · Coverage: 4/4 Zellen (100 %) bei `min_replicates=5`.

Komplexität und Code-Qualität (alle kleiner = besser; alle vier Zellen 100 % korrekt):

Metrik (Richtung)	CC thinking	CC no-think	pi thinking	pi no-think
<code>cognitive_max</code> (↓)	5.0 🏆	5.6	9.6	8.2
<code>cognitive_avg</code> (↓)	3.17 🏆	3.87	5.57	7.4
<code>mccabe_avg</code> (↓)	2.16	2.11 🏆	2.9	3.13
Smell-Summe <code>smell_total</code> (↓)	2.2	2.0	3.4	1.2 🏆
Produktiv-LoC <code>lines_of_code</code> (↓)	44.6	40.0	35.2 🏆	42.2
Code-Mass (APP) <code>code_mass</code> (↓)	158.6	171.8	149.2 🏆	159.6
Korrektheit (innen) <code>tests_passing</code> (↑)	100 % 🏆	100 % 🏆	100 % 🏆	100 % 🏆
<code>duration_seconds</code> (↓)	718.6	579.2	339.4	318 🏆

Befunde:

- **F-1.1** — Der Harness-Effekt auf die Komplexität dominiert den Thinking-Effekt
- **F-1.2** — Thinking senkt die Code-Komplexität auf keinem der beiden Harnesses verlässlich
- **F-1.3** — pi ist knapper und schneller, CC komplexitätsärmer und smell-stabiler

Ein Zwei-Faktor-Test mit klarer Rangfolge: Der Harness-Pfad prägt das Komplexitäts-Profil stärker als die Reasoning-Stufe. Über beide Thinking-Stufen schreibt derselbe Opus auf dem einen Harness durchweg komplexitätsärmeren Code (`cognitive_max` ~5 gegen ~8–10), während der Thinking-Effekt innerhalb eines Harness kleiner und uneinheitlich bleibt. Bemerkenswert: Thinking wirkt weniger auf den Mittelwert als auf die Streuung — ohne Thinking treten dichte Einzelfunktions-Ausreißer auf (ein pi-Run legt die gesamte Logik in eine Funktion, `cognitive_avg` = 17 bei 27 LoC), dasselbe Dichte-Muster wie bei Opus auf dem cursor-Harness. Bindender Caveat: Harness und Workflow-Linie sind hier nicht separierbar (zwei Linien derselben Generation). [findings.md](#)

5. Cross-RQ-Synthese

1. **Die trainingsbekannte Kata misst fast nichts — und genau das macht sie als Kontrolle wertvoll.** Auf `game-of-life` erreichen in RQ-prompt-known-kata alle drei Prompt-Stile, in RQ-tdd-quality alle acht Workflows und in RQ-model-quality elf von zwölf Modell-Zellen dieselbe perfekte Korrektheit. Jeder Faktor, der auf der neuartigen Kata 70 Prozentpunkte bewegt, ist dort unsichtbar. Wer nur auf bekannten Aufgaben evaluiert, misst systematisch Null-Effekte und schließt daraus fälschlich, die Faktoren seien gleichwertig. Die Konsequenz für eigene Evaluationen: Der Benchmark muss Mehrdeutigkeiten enthalten, sonst ist er blind — und umgekehrt lässt sich die bekannte Kata gezielt einsetzen, um Korrektheit als Störgröße auszuschalten und reine Code-Qualität zu messen.
2. **Interne Tests sind als Abnahmekriterium wertlos — quer durch alle Modell- und Harness-RQs.** `tests_passing` liegt in RQ-model-quality, RQ-model-quality-oc, RQ-model-novel-oc, RQ-model-quality-pi und RQ-model-novel-pi fast durchgehend bei 100 %, während die externe Suite in denselben Zellen von 0.00 bis 1.00 spreizt. Die Muster dahinter sind verschieden und alle für Agenten typisch: ein Modell schreibt 30.8 grüne Tests zu einer selbstkonsistent falsch verstandenen Spec (RQ-model-novel-oc); ein anderes minimiert die Test-Liste und testet nur, was es ohnehin implementiert hat (RQ-model-quality-oc); ein drittes ignoriert eine ganze Spec-Operation, deren Tests es nie angelegt hat (RQ-model-novel). Ein Agent, der seine eigenen Tests schreibt, definiert damit auch seinen eigenen Erfolgsmaßstab. Praktisch heißt das: Abnahme braucht Kriterien, die der Agent nie gesehen hat.
3. **Jeder gefundene Sieger ist kontextgebunden — über drei unabhängige Achsen hinweg.** Das Workflow-Ranking kippt mit dem Modell (RQ-workflow-model: derselbe Workflow 0.93 auf einem Modell, 0.67 auf dem nächsten), mit der Kata (RQ-tdd-quality: Platz 1 auf der einen, Platz 8 auf der anderen) und mit dem Harness (RQ-model-quality-cc-vs-pi: dasselbe Modell trägt komplexitätsärmeren Code auf dem einen Pfad). Keine der drei Achsen lässt sich aus den anderen vorhersagen. Der Befund ist unbequem, weil er die Übertragbarkeit von Empfehlungen begrenzt — aber er ist konsistent über alle betroffenen RQs und die praktische Lehre daraus ist eindeutig: Eine Workflow-Empfehlung ohne Angabe von Modell, Aufgabentyp und Werkzeug ist keine Empfehlung.
4. **Was die Prompt-Struktur an Verhalten steuert, steuert sie über den Refactor-Aufruf — nicht über die Formulierung von Sorgfalt.** Der Vergleich in RQ-pocock-vs-v62 zeigt es am schärfsten: Ein externer TDD-Skill, der Refactoring als Nachgedanken formuliert („nach den grünen Tests nach Kandidaten schauen“), löst in drei von drei Läufen exakt null Refactorings aus und landet bei `cognitive_max` 14.3 — gegen 5.0 der Variante mit Refactor-Phase pro Zyklus. Dieselbe Mechanik erklärt den Harness-Unterschied in RQ-harness-requesty: Der Harness mit der niedrigsten Spitzen-Komplexität ist derjenige, der den Refactor-Schritt rund dreimal so oft tatsächlich aufruft. Nicht die Erwähnung von Qualität im Prompt erzeugt Qualität, sondern die erzwungene Ausführung des Schritts.

5. Die Kosten-Achse ist weitgehend entkoppelt von der Ergebnis-Achse — und das ist eine praktische Chance. In RQ-cost-sol-pi-vs-opus-cc spart der Wechsel auf ein günstigeres Modell-Harness-Bündel 68–92 % der Kosten bei gleicher oder besserer Korrektheit; in RQ-harness-requesty spreizen die Kosten über vier Harnesse um Faktor 3.5, ohne die Korrektheit zu bewegen; in RQ-model-quality-oc löst das günstigste Modell die Aufgabe für ein Achtzehntel des Preises des teuersten. Was der Aufpreis kauft, ist in allen drei Fällen dasselbe und nur dasselbe: niedrigere Komplexität und weniger Smells. Wer ein korrektes Ergebnis braucht und die Wartbarkeit nicht — Skripte, Glue-Code, Wegwerf-Prototypen — zahlt derzeit routinemäßig ein Vielfaches für einen Vorteil, den er nicht nutzt.

6. Limitierungen

- **Nur TypeScript.** Alle Runs laufen im selben pnpm/tsx/Vitest/ESLint+SonarJS-Stack. Die tragenden Qualitäts-Metriken (`cognitive_max`, `mccabe_max`, `smell_total`) sind an dieses Werkzeug-Ökosystem gebunden; ob die Workflow-Effekte in Python, Go oder Java gleich ausfallen, ist ungeprüft.
- **Nur synthetische Katas, und nur zwei tragen die Hauptlast.** `game-of-life` (~30–40 Produktiv-LoC) und `claim-office` (~150–320) liefern den Großteil der Evidenz. Die Kata-Achse ist damit auf „klein und trainingsbekannt“ gegen „mittelgroß und neuartig“ reduziert. Bestandscode, Web-Anwendungen, Datenbank- und Async-Systeme kommen nicht vor — gerade dort, wo Refactoring am teuersten ist, fehlen Daten.
- **Headless, ohne Human-in-the-Loop.** Die Zahlen beschreiben unbeaufsichtigte Autonomie. Mehrere der dokumentierten Korrektheits-Verluste sind Fehlermodi, die eine einzige menschliche Rückfrage abfinge: vorzeitiger Selbst-Abbruch, unvollständige Test-Listen, falsch geratene CLI-Verträge. Für interaktive Nutzung sind die Werte damit eine untere Schranke, keine Vorhersage.
- **Kleine Zellen.** Der Standard liegt bei $n=5$, mehrere Zellen bei $n=3$. Aus der Stabilitäts-RQ ist bekannt, dass bei $n=3$ die vollständige Workflow-Rangordnung nur in 16–63 % der Subsamples korrekt reproduziert wird — belastbar sind große Effekte, nicht marginale Unterschiede. Wo im Text Rangfolgen mit engem Abstand stehen, sind sie entsprechend als vorläufig zu lesen.
- **Kosten sind Listenpreis-Schätzungen, keine Abrechnungen.** Alle `cost_usd`-Werte entstehen aus gemessenen Tokens $\times$ öffentlichem Tarif. Rabatte, Smart-Routing und workspace-spezifische Konditionen fehlen. Zusätzlich tragen die Harnesse teils verschiedene Tarif-Kanäle (nativer Listpreis gegen Gateway-Aufschlag) — ein Teil der Harness-Kostendifferenz ist Tarif-, nicht Effizienz-Effekt.
- **Routing-Konfundierung zwischen Modell-Generationen.** Ein Teil der Modelle lief über Portkey, ein Teil über Requesty, ein Teil nativ. Wo Zellen verschiedener Routing-Wege verglichen werden, ist der Effekt nicht sauber vom Routing trennbar; die betroffenen RQ-Abschnitte weisen das jeweils aus.
- **Eine Coverage-Lücke.** Alle 19 berichteten Forschungsfragen sind vollständig gefüllt (100 % der Zellen bei der jeweiligen `min_replicates`-Schwelle). Die einzige Ausnahme liegt in der hier ausgesparten Workflow-Entwicklungs-Linie.
- **Der Modell-Vergleich ist eine Momentaufnahme.** Die Modell-Landschaft bewegt sich schneller als die Erhebung: Zwischen den frühesten und den jüngsten Runs dieses Snapshots sind mehrere Modell-Generationen erschienen. Absolute Modell-Rankings altern entsprechend schnell; die Struktur-Befunde (welcher Faktor welchen Outcome treibt) sind deutlich stabiler als die Namen in den Tabellen.

7. Reproduzierbarkeit

Alle Daten und Analyse-Skripte liegen im Repo:

- `research/questions-{claude,opencode,pi,cursor-cli,cross}/*/README.md` — RQ-Definitionen (Frontmatter mit factors/controls/outcomes)
- `research/questions-{claude,opencode,pi,cursor-cli,cross}/*/findings.md` — persistente Befund-Listen
- `research/workflow-dev/*` — die hier ausgesparte Workflow-Entwicklungs-Linie (13 RQs, 272 Runs), gleiche Struktur
- `experiments/runs/*/metrics.json` — Rohdaten pro Run
- `experiments/aggregate-by-query.py` — RQ-spezifische Aggregation
- `experiments/batch-plan-from-rq.py` — Batch-Plan-Generierung aus RQ-Frontmatter
- `experiments/docker/Dockerfile` + `run-batch.sh` + `batch.sh` — Container-Pipeline
- `experiments/analyze-run.sh` + `analyze_transcript.py` — Run-Analyse

Container-Pins: `claude-code@2.1.170`, `opencode-ai@1.15.10`, `@earendil-works/pi-coding-agent@0.81.1`, `pnpm@9.15.9` (siehe `experiments/docker/Dockerfile`).

8. Files

Pfad	Inhalt
<code>research/questions-claude/1.1-prompt-style-correctness/findings.md</code>	RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?
<code>research/questions-claude/1.1-prompt-style-correctness/runs.csv</code>	RQ-prompt-correctness aggregierte Run-Metriken
<code>research/questions-claude/1.2-prompt-style-known-kata/findings.md</code>	RQ-prompt-known-kata — Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?
<code>research/questions-claude/1.2-prompt-style-known-kata/runs.csv</code>	RQ-prompt-known-kata aggregierte Run-Metriken
<code>research/questions-claude/2.1-model-effect-code-quality/findings.md</code>	RQ-model-quality — Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8, Fable 5 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?

Pfad	Inhalt
research/questions-claude/2.1-model-effect-code-quality/runs.csv	RQ-model-quality aggregierte Run-Metriken
research/questions-claude/2.2-model-effect-novel-kata/findings.md	RQ-model-novel — Wie unterscheiden sich Fable 5, Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?
research/questions-claude/2.2-model-effect-novel-kata/runs.csv	RQ-model-novel aggregierte Run-Metriken
research/questions-claude/3.1-workflow-model-interaction/findings.md	RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?
research/questions-claude/3.1-workflow-model-interaction/runs.csv	RQ-workflow-model aggregierte Run-Metriken
research/questions-claude/4.1-tdd-effect-code-quality/findings.md	RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualitaet aus, und macht die TDD-Striktheit einen Unterschied?
research/questions-claude/4.1-tdd-effect-code-quality/runs.csv	RQ-tdd-quality aggregierte Run-Metriken
research/questions-claude/4.2-tdd-effect-correctness/findings.md	RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?
research/questions-claude/4.2-tdd-effect-correctness/runs.csv	RQ-tdd-correctness aggregierte Run-Metriken
research/questions-claude/4.3-tdd-context-engineering/findings.md	RQ-context — Welche Form der Kontext-Strukturierung — isolierte Subagent-Kontexte pro TDD-Phase (v4.1), ein geteilter, akkumulierter Single-Context (v5.1), ein Hybrid mit Skill-basiertem Red/Green im Shared-Context und isoliertem Refactor-Subagent (v6.1) oder ein Hybrid mit isolierten Green- und Refactor-Subagents bei Shared-Context-Test-Liste/Red (v7.1) — führt zu besserer Code-Qualitaet?
research/questions-claude/4.3-tdd-context-engineering/runs.csv	RQ-context aggregierte Run-Metriken
research/questions-claude/4.4-external-tdd-pocock-vs-v62/findings.md	RQ-pocock-vs-v62 — Wie schneidet der externe Matt-Pocock-TDD-Skill (v9-pocock-tdd: Single-Skill, Inline-Phasen, Tail-Refactor) auf claim-office-example-mapping gegen die interne Default-Baseline v6.2-with-why-cleaned (Multi-Command + Refactor-Subagent, Per-Cycle-Refactor) ab — auf Korrektheit, Code-Qualitaet, TDD-Disziplin und Kosten?
research/questions-claude/4.4-external-tdd-pocock-vs-v62/runs.csv	RQ-pocock-vs-v62 aggregierte Run-Metriken
research/questions-claude/5.1-workflow-stability/findings.md	RQ-stability — Wie stabil sind Code-Qualitaet und TDD-Disziplin pro Workflow ueber Replikate, und unter welchen Bedingungen ist n=3 als Replikat-Anzahl ausreichend?
research/questions-claude/5.1-workflow-stability/runs.csv	RQ-stability aggregierte Run-Metriken
research/questions-opencode/1.1-model-quality-oc/findings.md	RQ-model-quality-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle (Opus 4.7 via Portkey + vier Nicht-Anthropic-Modelle aus dem Portkey-Catalog) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?
research/questions-opencode/1.1-model-quality-oc/runs.csv	RQ-model-quality-oc aggregierte Run-Metriken
research/questions-opencode/1.2-model-novel-kata-oc/findings.md	RQ-model-novel-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?
research/questions-opencode/1.2-model-novel-kata-oc/runs.csv	RQ-model-novel-oc aggregierte Run-Metriken
research/questions-pi/1.1-model-quality-pi/findings.md	RQ-model-quality-pi — Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v6.2.1-phase-continuation-pi-Workflow?
research/questions-pi/1.1-model-quality-pi/runs.csv	RQ-model-quality-pi aggregierte Run-Metriken
research/questions-pi/1.2-model-novel-kata-pi/findings.md	RQ-model-novel-pi — Wie unterscheiden sich die via pi-Harness (Requesty-Routing) erreichbaren Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v6.2-with-why-cleaned-pi-Workflow?
research/questions-pi/1.2-model-novel-kata-pi/runs.csv	RQ-model-novel-pi aggregierte Run-Metriken
research/questions-cursor-cli/1.1-model-quality-cursor/findings.md	RQ-model-quality-cursor — Wie unterscheiden sich die via cursor-cli-Harness erreichbaren Modelle (Opus, Composer, Grok) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping?
research/questions-cursor-cli/1.1-model-quality-cursor/runs.csv	RQ-model-quality-cursor aggregierte Run-Metriken
research/questions-cross/1.1-harness-effect/findings.md	RQ-harness — Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität und TDD-Disziplin aus, wenn Modell, Workflow-Intention und Prompt-Stil konstant gehalten werden?
research/questions-cross/1.1-harness-effect/runs.csv	RQ-harness aggregierte Run-Metriken
research/questions-cross/1.2-harness-requesty/findings.md	RQ-harness-requesty — Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität, TDD-Disziplin und Kosten aus, wenn Modell (opus-4-8 über Requesty), Workflow-Intention und Prompt-Stil konstant gehalten werden?
research/questions-cross/1.2-harness-requesty/runs.csv	RQ-harness-requesty aggregierte Run-Metriken
research/questions-cross/1.3-cost-sol-pi-vs-opus-cc/findings.md	RQ-cost-sol-pi-vs-opus-cc — Wie viel günstiger ist das GPT-Modell gpt-5-6-sol auf dem pi-Harness gegenüber opus-4-8 auf Claude Code — bei gleichem Prompt-Stil und outcome-äquivalentem TDD-Workflow, über beide Katas?
research/questions-cross/1.3-cost-sol-pi-vs-opus-cc/runs.csv	RQ-cost-sol-pi-vs-opus-cc aggregierte Run-Metriken
research/questions-cross/1.4-opus-cc-vs-pi/findings.md	RQ-model-quality-cc-vs-pi — Unterscheidet sich das Code-Qualitäts-Profil von Opus (opus-4-8) zwischen dem Claude-Code- und dem pi-Harness, je mit und ohne Thinking, bei konstanter Workflow-Generation (v6.2)?
research/questions-cross/1.4-opus-cc-vs-pi/runs.csv	RQ-model-quality-cc-vs-pi aggregierte Run-Metriken
experiments/runs/	Alle Run-Verzeichnisse mit Source, Transcript, Metriken